

CHAPTER 1

INTRODUCTION

1.1 OBJECTIVE:

The primary objective of this project is to design and develop an intelligent Ad Click Fraud Detection System that can accurately identify and classify fraudulent and genuine ad clicks using machine learning techniques. The system aims to analyze large volumes of click data and detect abnormal patterns associated with bot-driven or malicious activities, thereby reducing financial losses for advertisers.

Another key objective is to perform effective feature engineering by extracting meaningful temporal and behavioral attributes such as click frequency, time gaps between clicks, IP-based statistics, and device-related information. These features help improve the predictive performance of machine learning models and enhance the system's ability to distinguish between legitimate user behavior and fraudulent activity.

The project also aims to implement and evaluate supervised learning algorithms, including Random Forest and XGBoost, using standard performance metrics such as accuracy, precision, recall, F1-score, and AUC. Based on evaluation results, the best-performing model is selected and deployed for real-time prediction.

Additionally, the project focuses on developing a user-friendly dashboard that allows users to upload data, view predictions, and analyze model performance through visualizations. Overall, the objective is to create a scalable, reliable, and practical solution for detecting ad click fraud and improving trust and efficiency in online advertising systems.

1.2 OVERVIEW:

We introduced Ad Click Fraud Detection System that can accurately identify and classify fraudulent and genuine ad clicks using machine learning techniques.

1.3 INTRODUCTION:

Online advertising has become a dominant revenue model for digital platforms and businesses worldwide. With the rapid growth of online advertisements, the number of user interactions such as ad impressions and clicks has increased significantly. However, this growth has also led to a serious challenge known as ad click fraud, where automated bots, malicious scripts, or coordinated human activities generate illegitimate clicks on advertisements. Such fraudulent activities result in substantial financial losses for advertisers, distort campaign analytics, and reduce trust in digital advertising ecosystems.

Ad click fraud typically involves repeated or abnormal clicking behavior originating from the same IP address, device, or network within short time intervals. Traditional rule-based detection methods are often ineffective in identifying these complex and evolving fraud patterns, as fraudsters continuously modify their strategies to bypass static detection rules. As a result, there is a growing need for intelligent and adaptive systems capable of analyzing large volumes of click data and accurately distinguishing between genuine user behavior and fraudulent activity.

Machine learning techniques provide a promising solution to this problem by learning hidden patterns from historical click data. By leveraging temporal features, behavioral patterns, and IP-based statistics, machine learning models can effectively detect anomalies associated with fraudulent clicks. In this project, an Ad Click Fraud Detection System is developed using supervised machine learning algorithms such as Random Forest and XGBoost. The system performs feature engineering, model training, evaluation, and deployment through an interactive dashboard, enabling real-time analysis and decision-making.

The proposed system aims to improve the reliability and efficiency of online advertising by minimizing invalid clicks, optimizing ad spending, and enhancing transparency for advertisers and ad networks. By combining robust machine learning models with effective data visualization and analysis, the system offers a scalable and practical approach to combating ad click fraud in modern digital advertising environments.

CHAPTER 2

LITERATURE SURVEY

[1] TITLE: Detecting Click Fraud in Online Advertising Networks

YEAR: 2017

AUTHOR: N. Metwally, D. Agrawal, A. El Abbadi

DESCRIPTION:

This paper presents a comprehensive framework for detecting click fraud in online advertising networks, addressing the increasing challenges posed by fraudulent interactions with digital ads. The authors focus on analyzing user click behavior patterns to identify abnormal activity that may indicate the presence of automated bots or malicious actors attempting to manipulate ad metrics. The framework employs advanced statistical techniques to monitor multiple aspects of user behavior, including the frequency and timing of clicks, session durations, and deviations from typical interaction patterns observed across the platform. By establishing baseline norms for expected click behavior, the system is able to detect anomalies that signify potential fraudulent activity, such as repeated clicks from the same IP address, unusual session behavior, or abnormally high click volumes within a short timeframe. The approach emphasizes real-time monitoring, allowing advertisers and platforms to respond promptly to suspicious activity, minimizing both financial losses and skewed analytics data. The adaptive nature of the detection thresholds ensures that the system can evolve alongside emerging fraud strategies, overcoming the limitations of static rule-based methods that may fail to recognize new or sophisticated attack patterns.

LIMITATIONS:

- Cannot detect all advanced bots accurately.
- Depends heavily on past user behavior data.
- May give false alerts for genuine users.

[2] TITLE: Fraud Detection in Online Advertising Campaigns Using Machine Learning
YEAR: 2019
AUTHOR: J. Liu, Y. Wang, K. Zhang

DESCRIPTION:

This study investigates the application of supervised machine learning techniques to detect fraudulent clicks in digital advertising campaigns, offering a more robust alternative to traditional rule-based detection methods. The authors focus on two widely used classifiers—Random Forest and Gradient Boosting—due to their ability to handle large, complex datasets and capture non-linear relationships among features. The study emphasizes the importance of careful feature engineering, extracting and constructing relevant features such as IP address, device type, operating system, geographic location, timestamp, click frequency, and behavioral patterns of users interacting with ads. The extracted features are carefully selected and processed to enhance model performance, as the study demonstrates that proper feature engineering significantly boosts the ability of classifiers to discriminate between genuine and fraudulent clicks. By training the models on labeled datasets, the classifiers learn to recognize subtle patterns in click behavior that may indicate automated bot activity or repeated malicious clicks.

Model evaluation shows that both Random Forest and Gradient Boosting achieve high precision, recall, and overall predictive accuracy, effectively identifying complex and previously unseen fraud patterns. The study highlights that machine learning-based approaches outperform traditional static rule-based methods, particularly in handling evolving fraudulent strategies and large-scale click data.

LIMITATIONS:

- Requires large labeled datasets for training.
- Model accuracy depends on quality of features selected.
- Training machine learning models can take more time.

[3] TITLE: Real-Time Click Fraud Detection Using Behavioral Analysis
YEAR: 2020
AUTHOR: R. Agarwal, S. Kumar, P. Singh

DESCRIPTION:

This paper presents a real-time click fraud detection system that leverages detailed behavioral analysis of users interacting with online advertisements, providing a proactive and adaptive approach to safeguarding digital marketing campaigns. The system continuously monitors user actions, including mouse movements, click intervals, scrolling behavior, and session durations, to capture subtle indicators of fraudulent activity. By analyzing these behavioral patterns, the framework can differentiate between genuine human interactions and automated or bot-generated clicks, which often exhibit repetitive or unnatural sequences.

To ensure rapid and effective detection, the system integrates lightweight machine learning models capable of real-time inference, making it suitable for deployment on high-traffic advertising platforms without causing performance bottlenecks. The models are trained on labeled datasets representing normal and fraudulent user behaviors, enabling them to adapt dynamically as new forms of click fraud emerge. Experimental evaluations demonstrate that the proposed approach significantly reduces financial losses due to fraudulent activity while enhancing the reliability of campaign analytics by providing more accurate data on actual human engagement. This study highlights the importance of combining behavioral analytics with automated learning techniques, offering a robust and scalable framework for click fraud detection. By focusing on real-time adaptability and minimal computational overhead, the approach provides advertisers with actionable insights and practical tools to maintain campaign integrity in rapidly evolving online environments.

LIMITATIONS:

- Requires continuous monitoring of user behavior.
- Collecting behavioral data may raise privacy concerns.
- Real-time analysis needs fast processing systems.

[4] TITLE: Click Fraud Detection with Ensemble Learning

YEAR: 2021

AUTHOR: L. Chen, H. Zhao, M. Sun

DESCRIPTION:

This paper presents an advanced ensemble learning framework designed to enhance the accuracy and robustness of ad click fraud detection. The authors combine multiple machine learning models—including Random Forest, XGBoost, and logistic regression—into a single predictive system, leveraging the unique strengths of each model. Random Forest captures complex non-linear relationships and patterns in user click behavior, XGBoost efficiently handles large datasets with high-dimensional features, and logistic regression provides a simple, interpretable baseline. By aggregating the outputs of these diverse models, the ensemble approach improves detection performance, reducing both false positives and false negatives compared to individual classifiers.

The framework focuses on analyzing multiple facets of user behavior, such as temporal click sequences, device fingerprinting, session activity, and anomalies in IP addresses or geographic patterns. By combining these heterogeneous features, the system can identify both conventional and emerging fraud strategies that often evade traditional detection methods. The authors also highlight the interpretability of the ensemble by analyzing feature importance scores, revealing which attributes—such as click frequency, device type, or unusual session intervals—contribute most to fraud detection.

LIMITATIONS:

- Ensemble models are complex to design and manage.
- Requires high computational power and memory.
- Training multiple models increases processing time.

[5] TITLE: Explainable Machine Learning for Ad Click Fraud Detection

YEAR: 2022

AUTHOR: M. R. Khan, S. T. Ahmed, A. Hussain

DESCRIPTION:

This paper investigates the application of explainable artificial intelligence (XAI) techniques to the domain of ad click fraud detection, with a focus on making predictive models more transparent and interpretable. The authors employ machine learning models such as Random Forest and XGBoost, and integrate SHAP (SHapley Additive exPlanations) to quantify the contribution of individual features toward each prediction. By analyzing feature-level explanations, the system enables advertisers and analysts to understand why a specific click is classified as legitimate or fraudulent, fostering trust in automated decision-making processes.

The study emphasizes the critical role of interpretability in real-world deployment, as opaque models can limit confidence in fraud detection systems despite high accuracy. Beyond model transparency, the research also investigates the effects of feature selection, data preprocessing, and input normalization on both predictive performance and explanation clarity. This ensures that the system is not only accurate but also actionable, guiding users toward understanding the behavioral patterns driving fraudulent clicks. Experimental results show that incorporating SHAP explanations helps identify the most influential features, such as unusual click timing, device anomalies, or repeated IP patterns, which are often indicative of bot-driven activity.

Overall, the paper demonstrates that explainable AI approaches enhance both the usability and reliability of ad click fraud detection dashboards.

LIMITATIONS:

- Explainable AI methods increase model complexity.
- SHAP analysis can require high computation time.
- Large datasets are needed for accurate explanations.

[6] TITLE: A Survey on Ad Fraud Detection Techniques

YEAR: 2023

AUTHOR: A. Gupta, P. Roy, S. Mehta

DESCRIPTION:

This paper provides an in-depth and comprehensive review of existing methods for detecting ad click fraud, encompassing a broad spectrum of strategies, from traditional rule-based and statistical techniques to modern machine learning-based approaches. The authors categorize these methods based on their detection mechanisms, data requirements, computational efficiency, and feasibility for deployment in real-world online advertising platforms. By doing so, the survey offers a structured understanding of how each approach addresses the challenges of identifying fraudulent clicks, including detecting anomalies, recognizing bot behaviors, and distinguishing between human and non-human traffic.

The survey also discusses emerging trends and innovative techniques that have shown promise in overcoming these challenges. Deep learning models, for instance, can automatically extract complex patterns from high-dimensional click data, while anomaly detection frameworks focus on identifying unusual behavior that may indicate fraudulent activity. Explainable AI (XAI) techniques are increasingly integrated to provide transparency in model decisions, enabling advertisers to understand the contributing factors behind each prediction. Additionally, the paper underscores the importance of combining multiple approaches, including hybrid models that integrate statistical rules with machine learning classifiers, to achieve better accuracy and robustness.

LIMITATIONS:

- Mostly reviews existing methods without proposing a new system.
- Some surveyed techniques may become outdated quickly.
- Deep learning methods require high computational resources.

[7] **TITLE:** Online Ad Click Fraud Detection Using Graph-Based Methods

YEAR: 2024

AUTHOR: K. Sharma, R. Patel, V. Kumar

DESCRIPTION:

This research presents a novel approach to ad click fraud detection through graph-based modeling, where users, devices, and advertisement interactions are represented as nodes and edges within a network graph. By structuring click data in this manner, the system is able to capture complex relationships and interactions that are often invisible to traditional feature-based methods. Fraudulent patterns, such as coordinated click campaigns or bot networks, are detected using advanced graph analytics techniques, including community detection, centrality analysis, and anomaly scoring. These methods allow the identification of clusters of suspicious activity and highlight nodes (users or devices) that exhibit abnormal connectivity or interaction patterns.

One of the key advantages of this graph-based approach is its ability to uncover hidden fraud structures in large-scale advertising networks. While conventional machine learning models analyze individual click attributes like IP address, device type, and timestamp, graph analytics considers the relationships between entities, providing a complementary perspective. Experimental evaluation demonstrates that graph-based techniques significantly improve detection accuracy, especially for organized fraudulent campaigns, and reduce false positives by contextualizing click activity within the broader network. Furthermore, integrating this approach into an ad click fraud dashboard enhances analytical capabilities, offering network-level insights that help advertisers understand not only which clicks.

LIMITATIONS:

- Graph-based models are complex to design and implement.
- Requires high memory and computational resources.
- Processing large network graphs can be time-consuming.

[8] TITLE: Behavioral Biometrics-Based Ad Click Fraud Detection

YEAR: 2021

AUTHOR: S. Kumar, R. Sharma, P. Mehta

DESCRIPTION:

This research focuses on leveraging behavioral biometrics to enhance the detection of fraudulent clicks in online advertising platforms. Behavioral biometrics analyze subtle, user-specific interaction patterns, such as mouse movements, scrolling speed, click intervals, dwell time on ads, and overall session dynamics. Unlike traditional methods that rely primarily on static identifiers like IP addresses, device fingerprints, or user agents, this approach captures real-time human-like interactions, which are inherently difficult for automated bots or click farms to replicate accurately.

The system first collects raw interaction data and extracts meaningful features that quantify user behavior. Key features include the timing between consecutive clicks, velocity and trajectory of cursor movement, scroll depth per session, and patterns of navigation across pages. These behavioral metrics are then processed through supervised machine learning classifiers, including Support Vector Machines (SVM), Random Forest, and XGBoost, which are trained to distinguish between legitimate human clicks and various forms of fraudulent activity.

LIMITATIONS:

- Collecting behavioral biometric data may raise privacy concerns.
- Requires continuous tracking of user activities.
- High-traffic platforms may face processing overhead.

[9] TITLE: Machine Learning Based Click Fraud Detection in Online Advertising

YEAR: 2021

AUTHOR: Pooja Gupta, Rakesh Sharma, Anil Kumar

DESCRIPTION:

This study presents a comprehensive machine learning framework specifically developed to detect click fraud within online advertising networks, with a focus on the Indian digital ecosystem where ad campaigns are growing rapidly. The authors analyze user click behavior by examining a range of features, including IP address frequency, device type, session duration, and click timing intervals, to identify anomalies that may indicate fraudulent activity. They implement and compare multiple supervised classifiers, such as Random Forest, Gradient Boosting, and Logistic Regression, to determine the most effective method for distinguishing between legitimate and fraudulent clicks.

The research underscores the critical role of feature selection and preprocessing, showing that carefully engineered features significantly enhance model performance by capturing subtle patterns of coordinated fraudulent behavior. Experimental results demonstrate that the Random Forest classifier achieves superior precision and recall, particularly in detecting complex bot attacks and repeated fraudulent interactions from the same sources. Beyond detection accuracy, the study emphasizes the importance of real-time monitoring, proposing adaptive thresholding and continuous evaluation of click streams to ensure timely identification of suspicious activity, which is vital for minimizing financial losses and maintaining the integrity of advertising analytics.

LIMITATIONS:

- Requires large labeled datasets for training.
- Accuracy depends on proper feature selection and preprocessing.
- Real-time monitoring needs high computational resources.

[10] TITLE: Real-Time Detection of Ad Click Fraud Using Ensemble Learning

YEAR: 2022

AUTHOR: Sneha Reddy, Manish Jain, Abhishek Verma

DESCRIPTION:

This paper presents an advanced ensemble learning-based framework for detecting fraudulent ad clicks, designed to leverage the strengths of multiple machine learning models simultaneously. The approach combines predictions from XGBoost, Random Forest, and Support Vector Machines (SVM), allowing the system to capture diverse and complex behavioral patterns exhibited by users during online ad interactions. Key behavioral features analyzed include click velocity, session duration, temporal click intervals, and anomalies in device fingerprints, enabling the detection of both automated bot activity and coordinated human-assisted fraud.

The authors demonstrate that ensemble methods provide superior performance compared to individual classifiers, achieving higher accuracy, F1-score, and robustness in the face of evolving and sophisticated click fraud tactics. In addition to predictive performance, the study incorporates feature importance analysis, which enhances interpretability by highlighting the specific click attributes that most significantly contribute to the classification of fraudulent activity. Practical experiments conducted on datasets derived from Indian online advertising campaigns validate the real-world applicability of the approach.

LIMITATIONS:

- Ensemble models require high computational power.
- Training multiple models increases processing time.
- Real-time deployment can be costly and complex.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM:

The existing fraud detection system uses **rule-based and manual filtering methods**, such as IP blocklists and fixed click limits. It relies on **static thresholds** and basic statistical techniques to identify fraudulent activities.

3.1.1 DISADVANTAGES:

- **No intelligent learning:** The system cannot learn from new data or improve over time.
- **No adaptability:** It fails to detect new or evolving bot behaviors.
- **High false positives:** Genuine user clicks may be wrongly marked as fraud.
- **Poor handling of imbalanced data:** Struggles when fraudulent cases are much fewer than normal ones.
- **Not real-time capable:** Cannot effectively detect fraud instantly as it happens.

3.2 PROPOSED SYSTEM:

The proposed system uses **Machine Learning–based ad fraud detection** to classify clicks as either **human or bot**. It applies advanced models like **Random Forest** and **XGBoost** to improve prediction accuracy. The system also performs **feature engineering**, extracting behavioral patterns such as click time gaps, IP click frequency, and night-time activity to better identify fraudulent behavior.

3.2.1 ADVANTAGE:

- **Handles imbalanced data:** Effectively detects rare fraud cases among large normal data.
- **Real-time prediction:** Uses a Flask web application to provide instant fraud detection.
- **Reduced errors:** Minimizes both false positives and false negatives.
- **Scalable & adaptable:** Can learn and adjust to new and evolving fraud patterns.
- **High accuracy & reliability:** Improves overall detection performance compared to traditional methods.

CHAPTER 4

SYSTEM REQUIREMENT

4.1 HARDWARE REQUIREMENTS:

The hardware requirements may serve as the basis for a contract for the implementation of the system and should therefore be a complete and consistent specification of the whole system. They are used by software engineers as the starting point for the system design. It shows what the system does and not how it should be implemented.

PROCESSOR	:	INTEL CORE I5 / I7 OR EQUIVALENT
RAM	:	MINIMUM 8 GB (RECOMMENDED 16 GB)
STORAGE	:	500 GB HDD OR SSD
GPU	:	NVIDIA GPU WITH CUDA SUPPORT

4.2 SOFTWARE REQUIREMENTS:

The software requirements document is the specification of the system. It should include both a definition and a specification of requirements. It is a set of what the system should do rather than how it should do it. The software requirements provide a basis for creating the software requirements specification. It is useful in estimating cost, planning team activities, performing tasks and tracking the team's and tracking the team's progress throughout the development activity.

OPERATING SYSTEM	:	WINDOWS 10 / 11 (64-BIT)
PROGRAMMING LANGUAGE	:	PYTHON 3.9+
FRAMEWORKS	:	TENSORFLOW, KERAS,

		SCIKIT-LEARN, TRANSFORMERS
LIBRARIES	:	NUMPY, PANDAS, OPENCV, SEABORN, MATPLOTLIB
DEVELOPMENT ENVIRONMENT	:	JUPYTER NOTEBOOK / ANCONDA
DEPLOYMENT	:	FLASK (FOR USER INTEFACE)

4.3 TECHNOLOGIES USED:

4.3.1 PYTHON:

Python is a powerful, flexible, and beginner-friendly language that plays a key role in modern AI & ML development. Its simplicity, strong libraries, and wide adoption make it the top choice for technical professionals.

4.3.1.1 Introduction of Python:

Python is a high-level, interpreted programming language known for its simplicity and readability. It was created by Guido van Rossum in 1991.

Python is widely used in:

- Web development
- Data analysis
- Automation
- Artificial Intelligence (AI)
- Machine Learning (ML)

It uses simple syntax, making it easy for beginners and powerful for professionals.

4.3.1.2 Work of Python (What Python Does):

Python is a versatile language used for multiple tasks:

- Automation: Automates repetitive tasks (scripts, bots)
- Web Development: Frameworks like Django, Flask
- Data Analysis: Libraries like Pandas, NumPy
- Artificial Intelligence & Machine Learning: Used to build models and predictions

4.3.1.3 Advantages of Python:

1. Simple and Readable Syntax

- Easy to learn and implement complex ML algorithms
- Reduces development time

2. Huge Library Support

Pre-built libraries for:

- Data processing
- Model building
- Visualization

3. Strong Community Support: Large community helps in debugging and learning

4. Integration Capability

Easily integrates with:

- Big Data tools (Hadoop, Spark)
- Cloud platforms

5. Scalability: Suitable for both small and large AI systems

6. Rapid Prototyping: Quickly test and deploy ML models
7. Platform Independent: Works on Windows, Linux, macOS

4.3.1.4 Role of Python in Prediction:

Python is used at every stage of a prediction system:

1. Data Collection: Read data from CSV, database, APIs
2. Data Preprocessing
 - Handle missing values
 - Normalize/scale data
 - Encode categorical variables
3. Feature Engineering: Create useful features (e.g., click frequency, time gap)
4. Model Building

Train ML models like:

- Random Forest
 - XGBoost
 - Linear Regression
 - Neural Networks
5. Model Evaluation: Accuracy, Precision, Recall, F1-score
 6. Prediction Output: Predict results (e.g., Fraud / Not Fraud)
 7. Deployment: Use frameworks like Flask to create web apps

4.3.1.5 Python Libraries Used:

Purpose	Library
Data Handling	Pandas, NumPy
Visualization	Matplotlib, Seaborn
ML Models	Scikit-learn
Deep Learning	TensorFlow, PyTorch
Deployment	Flask, Streamlit

4.3.2 FLASK (FOR USER INTERFACE):

4.3.2.1 What is Flask?

Flask is a lightweight web framework in Python used to build web applications and APIs.

In prediction projects, Flask is mainly used to:

- Create a User Interface (UI)
- Connect frontend with ML model
- Show prediction results to users

4.3.2.2 Role of Flask in Prediction:

Flask acts as a bridge between:

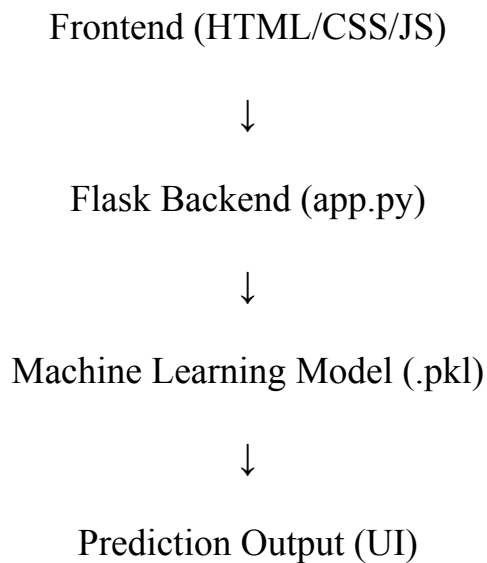
User Input (UI) ↔ ML Model ↔ Prediction Output

1. Workflow:

- User enters data in web form

- Flask receives input
- Flask sends data to ML model
- Model predicts result
- Flask displays output on webpage

2. Architecture of Flask-based Prediction App:



4.3.2.3 Advantages of Flask for UI:

1. Lightweight: Easy to set up and fast
2. Simple Integration: Easily connects ML models (like Random Forest, XG-Boost)
3. Flexible: No strict structure (you control everything)
4. REST API Support: Can be used for web & mobile apps
5. Real-Time Prediction: Instantly shows results after input

4.3.2.4 How It Works (Step-by-Step):

1. Run the Flask app
2. Open browser → `http://127.0.0.1:5000/`
3. Enter values in form
4. Click Predict
5. Output shown instantly

4.3.2.5 Where Flask is Used:

Based on your earlier fraud detection system:

- Input: Click time gap, IP frequency, activity time
- Model: Random Forest / XGBoost
- Output: Human / Bot classification
- Flask: Displays result in UI

4.4 JUPYTER NOTEBOOK / ANACONDA:

4.4.1 What is Jupyter Notebook?

Jupyter Notebook is a web-based tool used to write and run Python code interactively.

It allows you to:

- Write code in small blocks (cells)
- Run step-by-step execution
- Combine code + output + explanation
- Visualize data easily

4.4.2 What is Anaconda?

Anaconda is a distribution of Python that comes with:

- Pre-installed libraries (NumPy, Pandas, Scikit-learn)
- Package manager (conda)
- Tools like Jupyter Notebook

It simplifies environment setup for AI & ML projects.

4.4.3 Role in Prediction:

Jupyter Notebook Used for:

- Data analysis
- Data cleaning
- Feature engineering
- Model training
- Visualization
- Testing algorithms

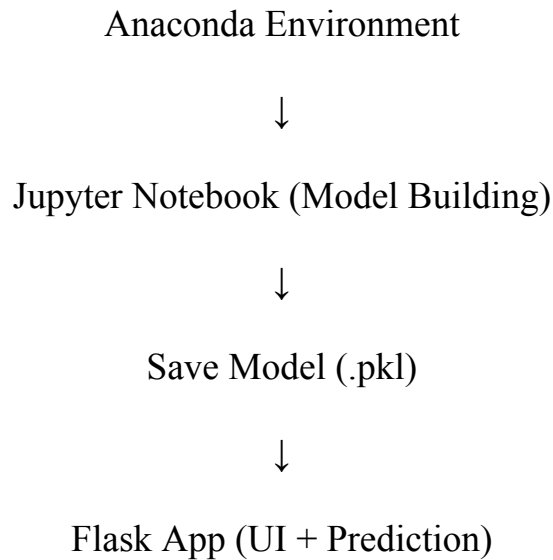
Example: Train your fraud detection model here

4.4.4 Anaconda Used for:

- Managing Python environments
- Installing ML libraries easily
- Avoiding dependency conflicts

Example: Create separate environment for your ML project

4.4.5 Workflow in Your Prediction Project:



4.4.6 Advantages of Jupyter Notebook:

1. Interactive Execution: Run code step-by-step
2. Easy Debugging: Quickly test and fix errors
3. Visualization Support: Graphs, charts, outputs in same place
4. Documentation: Combine notes + code

4.4.7 Advantages of Anaconda:

1. Easy Setup: No need to install libraries manually
2. Environment Management: Create isolated environments
3. Package Management: Install libraries easily
4. Stability: Reduces version conflicts

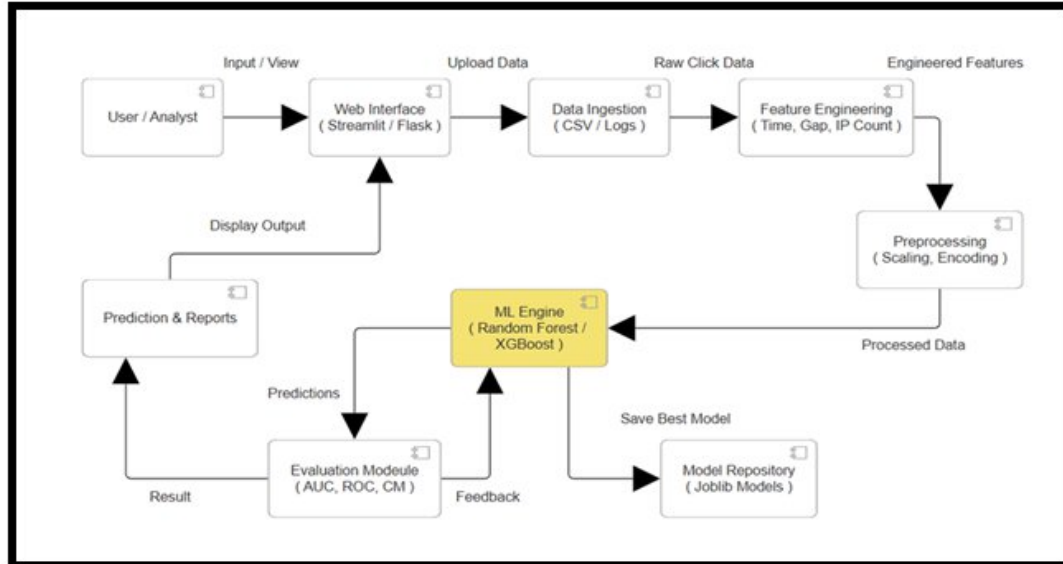
4.4.8 How They Work Together:

Tool	Purpose
Jupyter Notebook	Build & train model
Anaconda	Manage environment & libraries
Flask	Create UI & deploy model

CHAPTER 5

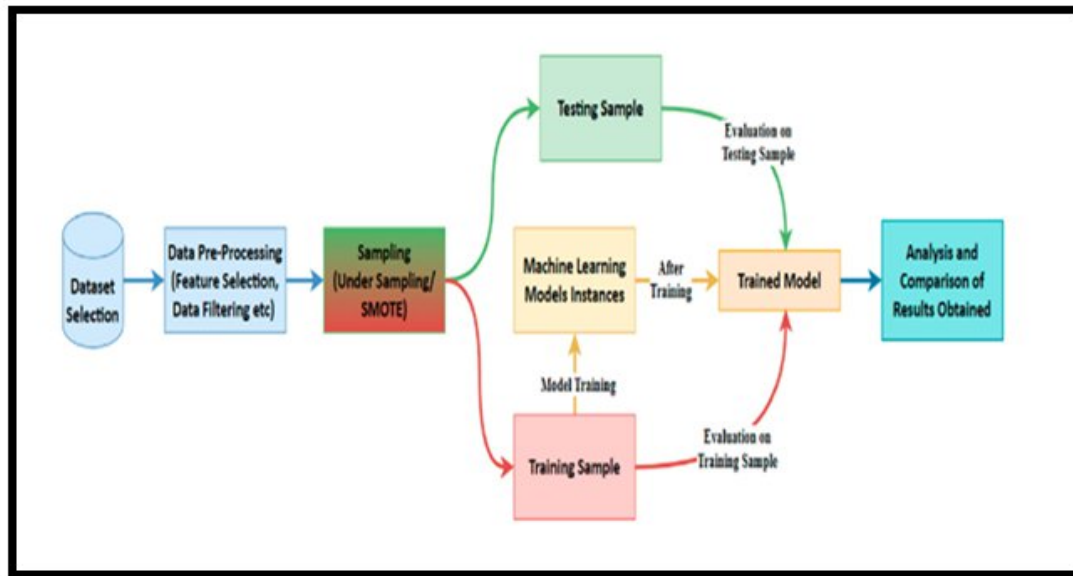
SYSTEM DESIGN SPECIFICATION

5.1 ARCHITECTURE DIAGRAM:



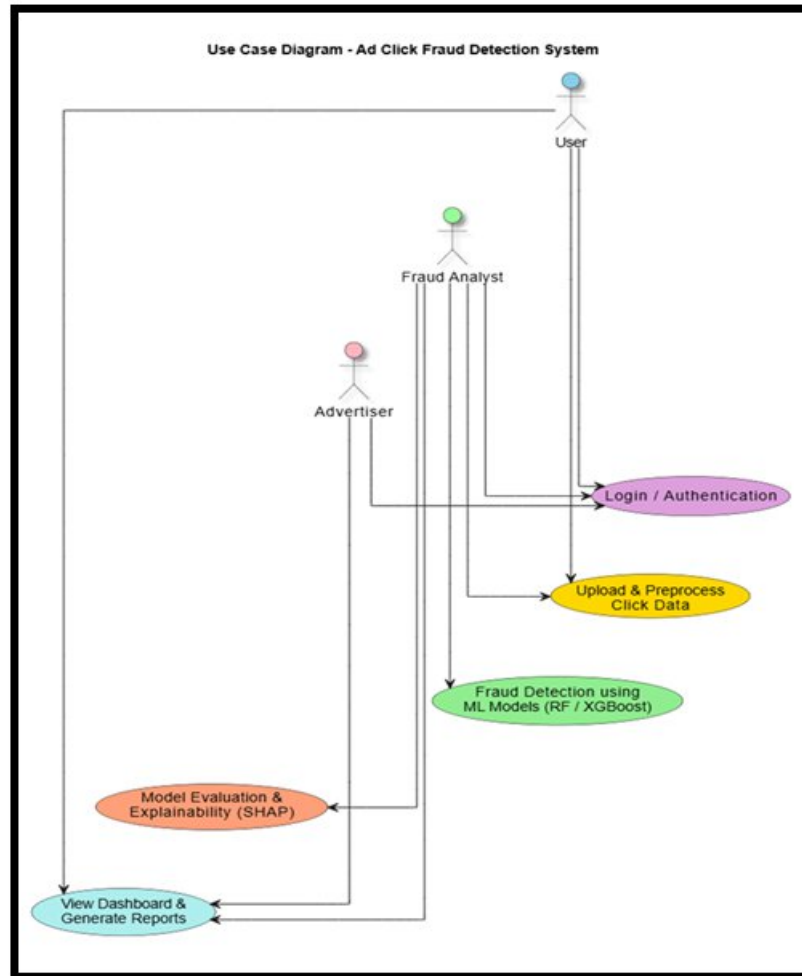
This diagram represents an end-to-end machine learning pipeline for a prediction system. The process begins with the user interacting through a web interface built using Flask or Streamlit, where data is uploaded in the form of CSV or logs. The system then performs data ingestion, followed by feature engineering (such as time gap and IP frequency) and preprocessing (scaling and encoding). The processed data is passed to the machine learning engine, which uses models like Random Forest and XGBoost to generate predictions. These predictions are evaluated using metrics like AUC, ROC, and confusion matrix, and the best model is saved for future use. Finally, the results are displayed to the user through the interface, with a feedback loop that helps improve model performance over time.

5.2 WORKFLOW DIAGRAM:



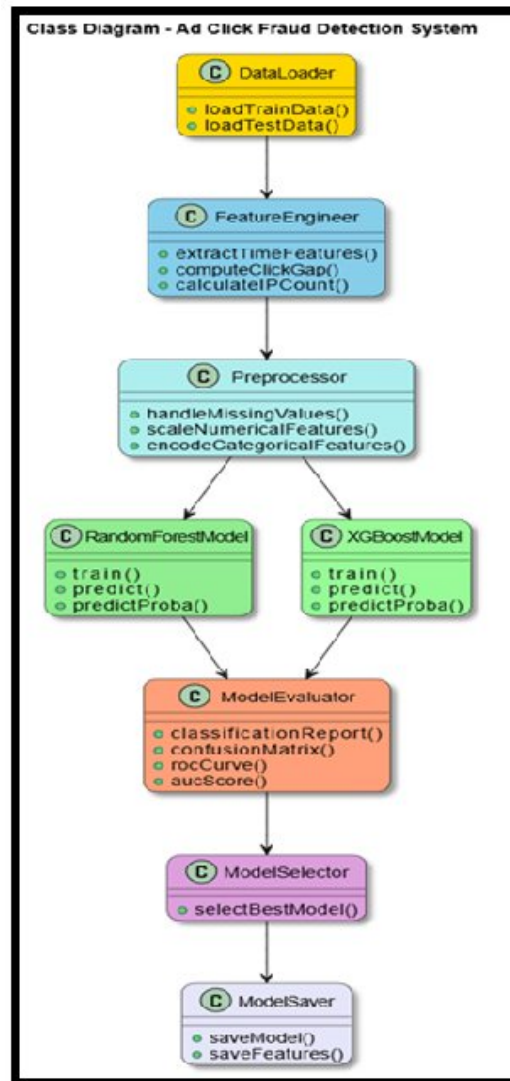
This diagram shows the machine learning workflow for building and evaluating a prediction model. The process starts with dataset selection, followed by data preprocessing such as feature selection and filtering. Then, sampling techniques like SMOTE or undersampling are applied to handle imbalanced data. The data is split into training and testing samples, where the training data is used to train machine learning models. After training, the model is evaluated on both training and testing datasets to measure performance. Finally, the trained model is analyzed, and results are compared to ensure accuracy and effectiveness of the prediction system.

5.3 USE CASE DIAGRAM:



This use case diagram represents the interaction between different actors in an ad click fraud detection system. The main users include the general user, advertiser, and fraud analyst, who access the system through login and authentication. They can upload and preprocess click data, which is then analyzed using machine learning models such as Random Forest and XGBoost for fraud detection. The system also provides model evaluation and explainability using techniques like SHAP, allowing better understanding of predictions. Finally, users can view dashboards and generate reports, enabling effective monitoring and decision-making in detecting fraudulent activities.

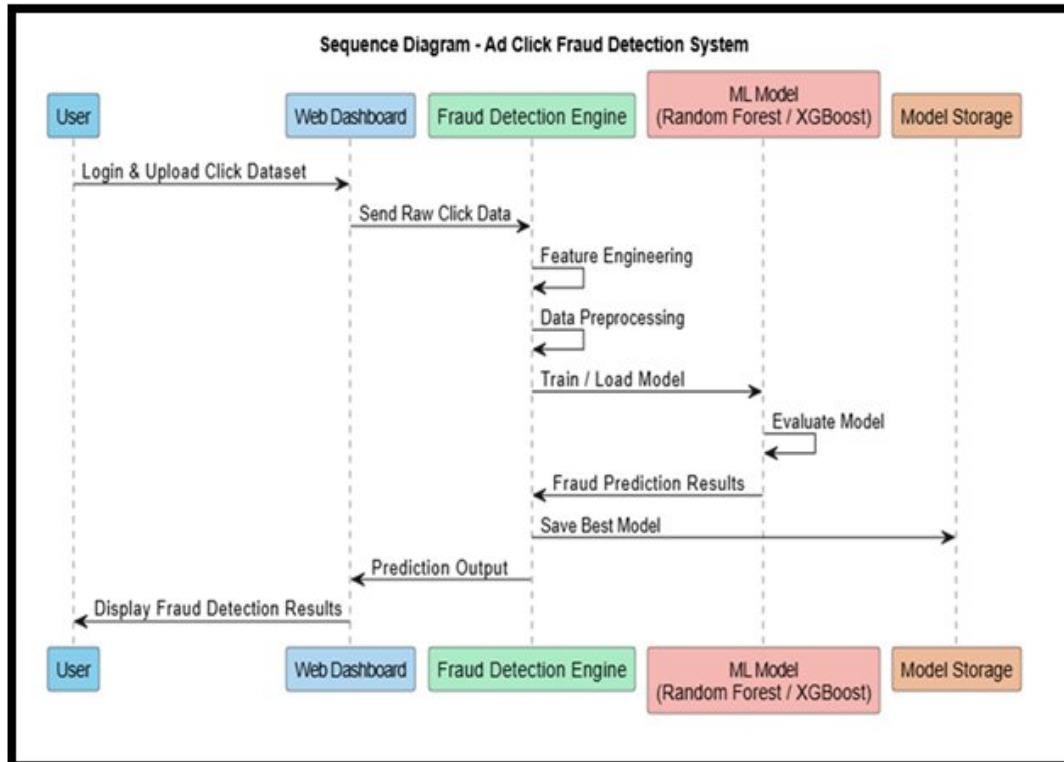
5.4 CLASS DIAGRAM:



This class diagram represents the structure of an ad click fraud detection system, showing how different components interact. The process begins with the **DataLoader** class, which loads training and testing data. The **FeatureEngineer** class then extracts useful features such as time patterns, click gaps, and IP counts. Next, the **Preprocessor** handles missing values, scales numerical data, and encodes categorical features. The processed data is passed to machine learning models like **Random Forest** and **XGBoost**, which perform training and prediction. The **ModelEvaluator** evaluates performance using metrics like confusion matrix, ROC curve, and AUC score. Based

on these results, the ModelSelector selects the best model, and finally, the ModelSaver stores the trained model and features for future use.

5.5 SEQUENCE DIAGRAM:



This sequence diagram illustrates the step-by-step interaction in the ad click fraud detection system. The process begins when the user logs in and uploads click data through the web dashboard, which forwards the raw data to the fraud detection engine. The engine performs feature engineering and preprocessing, then trains or loads machine learning models such as Random Forest and XGBoost. The model evaluates the data and generates fraud prediction results, while the best model is saved in storage. Finally, the prediction output is sent back to the dashboard and displayed to the user, completing the workflow.

CHAPTER 6

SYSTEM DESIGN IMPLEMENTATION

6.1 MODULE DESCRIPTION:

The system is structured into multiple functional modules, each performing a distinct and crucial role in the overall Ad Click Fraud Detection pipeline. These modules are designed to work in a seamless, interconnected manner, ensuring that every stage of the process—from data collection to final prediction—is handled efficiently and accurately. The **data acquisition module** gathers detailed click-related information from various online ad campaigns, capturing both legitimate user interactions and potential fraudulent activities. The **data preprocessing and feature engineering modules** clean, transform, and extract meaningful attributes from this raw data, enabling the machine learning models to detect subtle patterns indicative of fraud. The **model training and evaluation modules** ensure that predictive algorithms, such as Random Forest and XGBoost, are optimized and validated for accuracy, precision, and recall. Finally, the **deployment and dashboard modules** provide a real-time, user-friendly interface where predictions are visualized, and feature contributions are explained, giving stakeholders clear insights into click behavior. Together, these modules create a comprehensive, automated, and intelligent system capable of detecting fraudulent clicks in real-time, reducing financial losses, and supporting data-driven decision-making for advertisers.

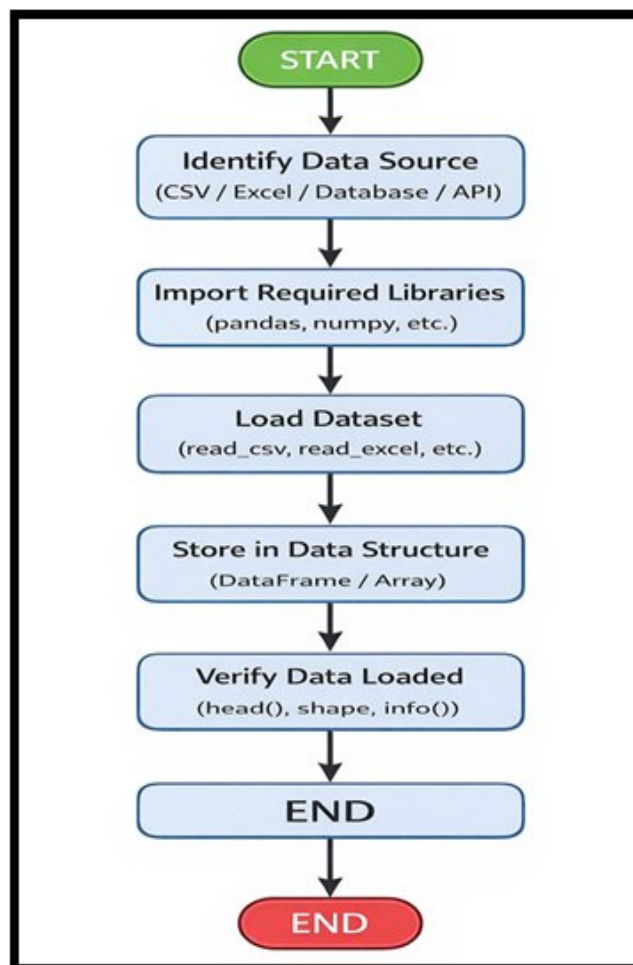
6.1.1 MODULES:

- DATA LOADING
- DATA ACQUISITION
- DATA PREPROCESSING
- DATA CLEANING
- MODEL TRAINING
- DATA SPLITTING

1. DATA LOADING:

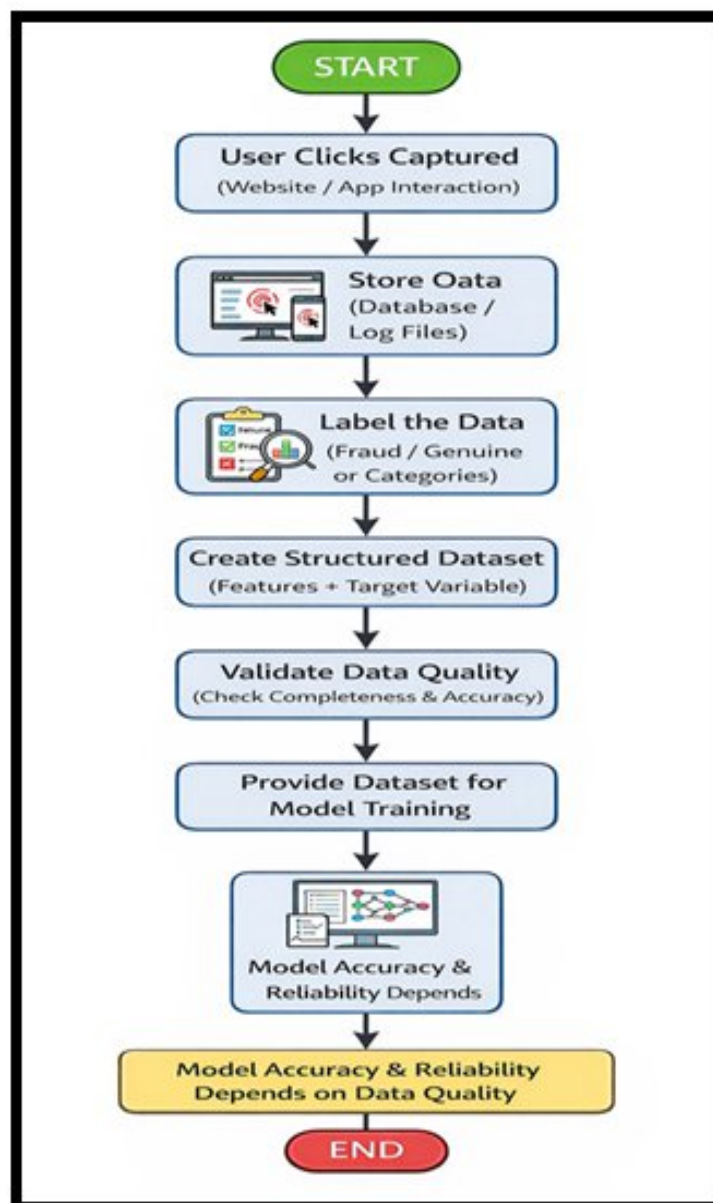
Data loading is the process of copying and loading data or data sets from a source file, folder or application to a database or similar application. It is usually implemented by copying digital data from a source and pasting or loading the data to a data storage or processing utility. Data loading is used in database-based extraction and loading techniques. Typically, such data is loaded into the destination application as a different format than the original source location.

For example, when data is copied from a word processing file to a database application, the data format is changed from .doc or .txt to a .CSV or DAT format. Usually, this process is performed through or the last phase of the Extract, Transform and Load (ETL) process. The data is extracted from an external source and transformed into the destination application's supported format, where the data is further loaded.



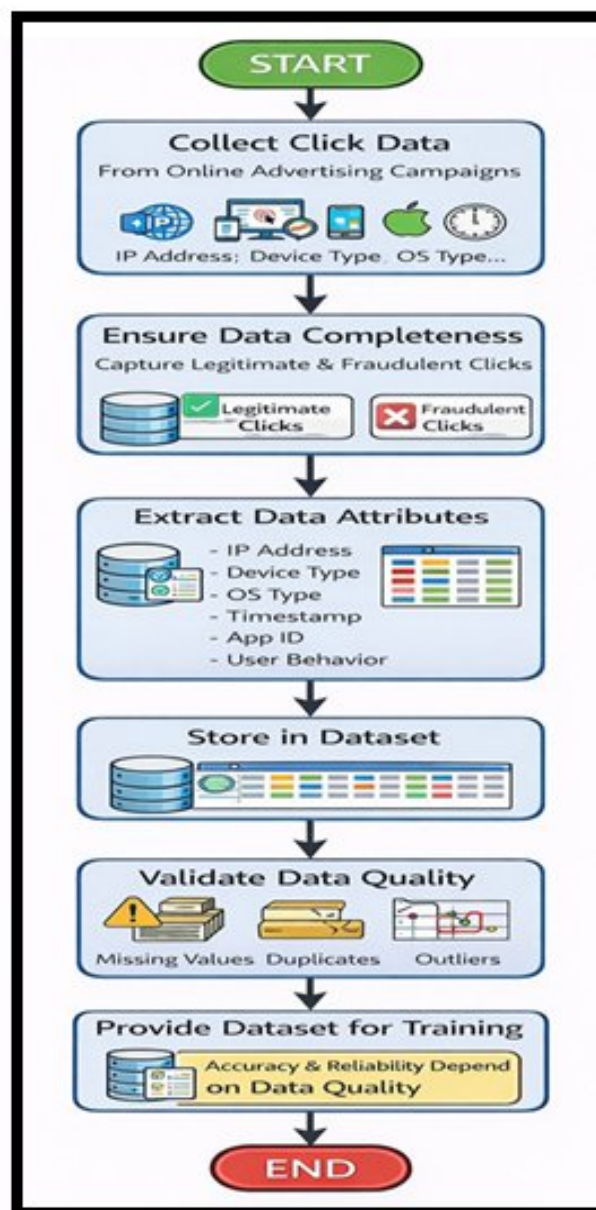
2. DATA ACQUISITION:

The data acquisition module is responsible for collecting click data from online advertising campaigns. This includes attributes such as IP addresses, device types, operating systems, timestamps, app identifiers, and user session behavior. The module ensures that both legitimate and fraudulent clicks are captured, providing a well-labeled dataset for model training. The quality and completeness of this data directly influence the accuracy and reliability of subsequent modules.



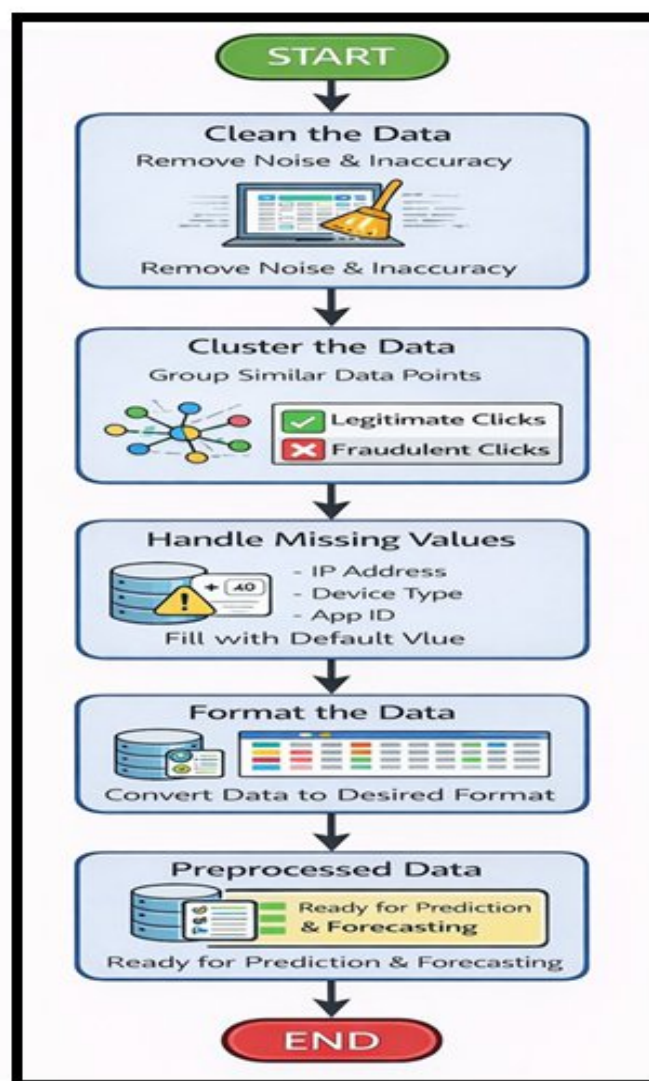
3. DATA PREPROCESSING:

Once the raw data is collected, the preprocessing module cleans and formats it for analysis. Missing values are handled through imputation, and categorical features are encoded into numeric formats suitable for machine learning algorithms. Noise and outliers are removed or treated, and features are normalized or scaled to ensure consistency across datasets. This module ensures that the models receive high-quality data, which is critical for accurate predictions.



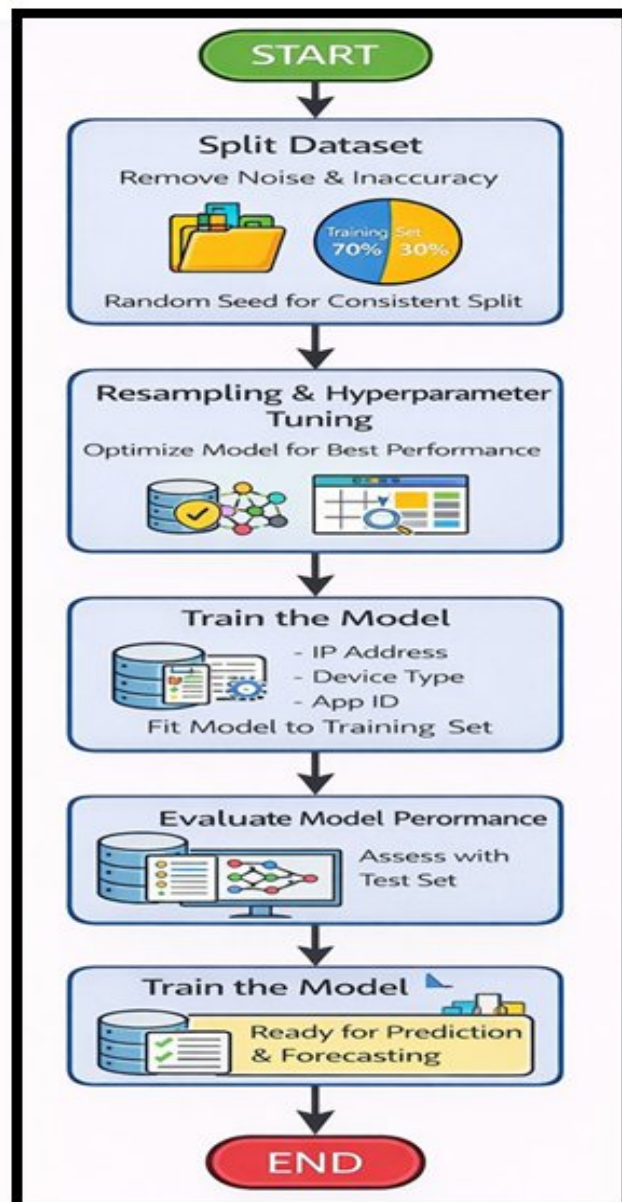
4. DATA CLEANING:

In this module the data is cleaned. After cleaning of the data, the data is grouped as per requirement. This grouping of data is known as data clustering. Then check if there is any missing value in the data set or not. If there is some missing value then change it by any default value. After that if any data need to change its format, it is done. That total process before the prediction is known as data pre-processing. After that the data is used for the prediction and forecasting step.



5. MODEL TRAINING:

For each experiment, we split the entire dataset into 70% training set and 30% test set. We used the training set for resampling, hyper parameter tuning, and training the model and we used test set to test the performance of the trained model. While splitting the data, we specified a random seed (any random number), which ensured the same data split every time the program executed.



6. DATA SPLITTING:

For each experiment, we split the entire dataset into 70% training set and 30% test set. We used the training set for resampling, hyper parameter tuning, and training the model and we used test set to test the performance of the trained model. While splitting the data, we specified a random seed (any random number), which ensured the same data split every time the program executed.

- **Post processing:**

After cleaning the data, data is normalized in training and testing the model. When data is splitted then we train algorithm on the training data set and keep test data set aside. This training process will produce the training model based on logic and algorithms and values of the feature in training data. Basically aim of feature extraction is to bring all the values under same scale.

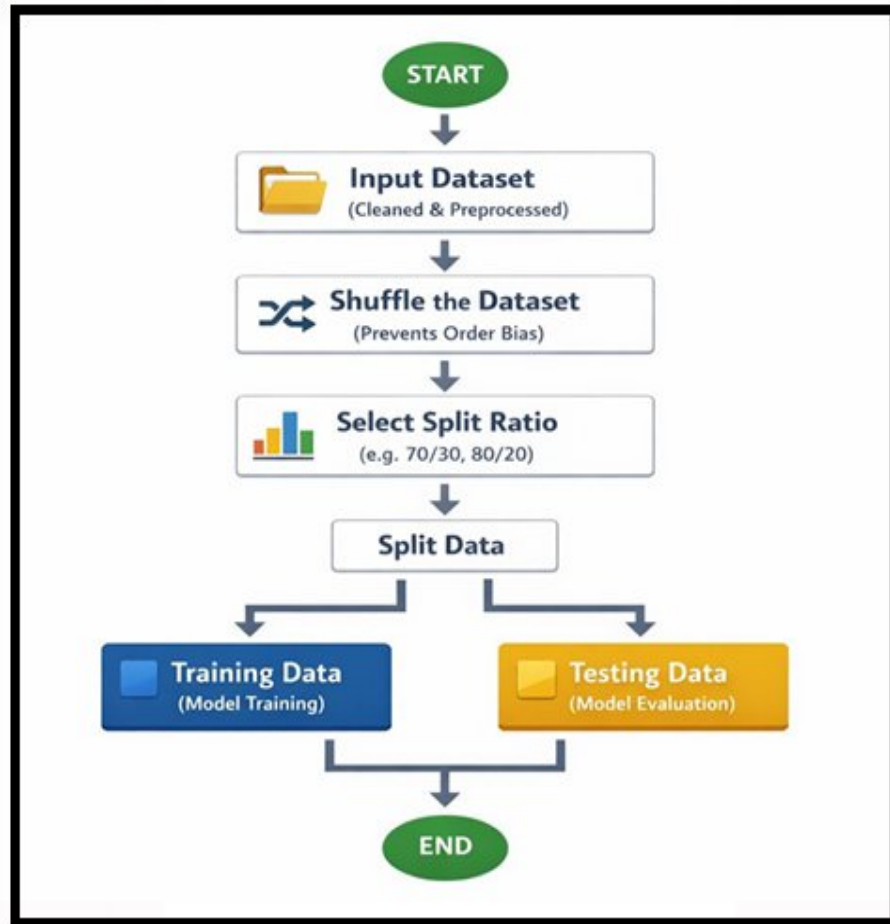
A dataset used for machine learning should be partitioned into three subsets — training, test, and validation sets.

Training set: -A data scientist uses a training set to train a model and define its optimal parameters — parameters it has to learn from data.

Test set: - A test set is needed for an evaluation of the trained model and its capability for generalization. The latter means a model's ability to identify patterns in new unseen data after having been trained over a training data. It's crucial to use different subsets for training and testing to avoid model over fitting, which is the incapacity for generalization we mentioned above.

- **Classification**

When data has been ready we apply Deep Learning Technique. We use different classification and ensemble techniques, to predict mental illness. The methods applied on brain MRI dataset. Main objective to apply Deep Learning Techniques to analyze the performance of these methods and find accuracy of them, and also been able to figure out the responsible/important feature which play a major role in prediction.



6.1.2 CLASSIFIER TRAINING:

A classifier is a function that takes features as input and generates a class label prediction. Based on the learning function and underlying assumptions, different types of classifiers can be developed. Neuroimaging studies have applied various classifiers for mental illness prediction. The dimensionality issue associated with the relatively large number of features and the small number of samples should be accounted for while applying such classification algorithms.

CHAPTER 7

MACHINE LEARNING & ALGORITHM TECHNIQUES:

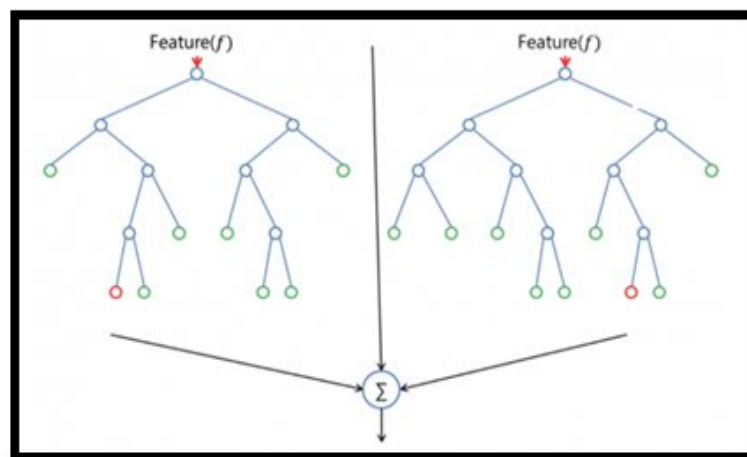
7.1 APPLYING MACHINE LEARNING:

7.1.1 Decision tree:

Decision tree is a supervised learning algorithm. The "forest" it builds, is an ensemble of decision trees, usually trained with the “bagging” method. The general idea of the bagging method is that a combination of learning models increases the overall result.

Put simply: decision tree builds multiple decision trees and merges them together to get a more accurate and stable prediction.

One big advantage of decision tree is that it can be used for both classification and regression problems, which form the majority of current machine learning systems. Let's look at decision tree in classification, since classification is sometimes considered the building block of machine learning. Below you can see how a decision tree would look like with two trees:



Decision tree has nearly the same hyper parameters as a decision tree or a bagging classifier. Fortunately, there's no need to combine a decision tree with a bagging classifier because you can easily use the classifier-class of decision tree. With decision tree, you can also deal with regression tasks by using the algorithm's regressor.

Decision tree adds additional randomness to the model, while growing the trees. Instead of searching for the most important feature while splitting a node, it searches for the best feature among a random subset of features. This results in a wide diversity that generally results in a better model.

Therefore, in decision tree, only a random subset of the features is taken into consideration by the algorithm for splitting a node. You can even make trees more random by additionally using random thresholds for each feature rather than searching for the best possible thresholds (like a normal decision tree does).

7.1.2 PERFORMANCE MATRICES:

Data was divided into two portions, training data and testing data, both these portions consisting 70% and 30% data respectively. All these two algorithms were applied on same dataset using Enthought Canaopy and results were obtained.

$$\text{Accuracy} = (TP + TN) / (P + N)$$

Predicting accuracy is the main evaluation parameter that we used in this work. Accuracy can be defined using equation. Accuracy is the overall success rate of the algorithm.

7.1.3 CONFUSION MATRIX:

It is the most commonly used evaluation metrics in predictive analysis mainly because it is very easy to understand and it can be used to compute other essential metrics such as accuracy, recall, precision, etc. It is an NxN matrix that describes the overall performance of a model when used on some dataset, where N is the number of class labels in the classification problem.

All predicted true positive and true negative divided by all positive and negative. True Positive (TP), True Negative (TN), False Negative (FN) and False Positive (FP) predicted by all algorithms are presented in table.

Negative (0)	True Negative (TN)	False Positive (FP)
	False Negative (FN)	True Positive (TP)
	Negative (0)	Positive (1)
	Predicted	

- True positive (TP) indicates that the positive class is predicted as a positive class, and the number of sample positive classes was actually predicted by the model.
- False negative indicates (FN) that the positive class is predicted as a negative class, and the number of negative classes in the sample was actually predicted by the model.
- False positive (FP) indicates that the negative class is predicted as a positive class, and the number of positive classes of samples was actually predicted by the model.
- True negative (TN) indicates that the negative class is predicted as a negative class, and the number of sample negative classes was actually predicted by the model.

7.1.4 ENSEMBLE AND PREDICTION MODULE:

To improve robustness and accuracy, predictions from multiple trained models are aggregated using ensemble techniques. This module combines the strengths of individual classifiers, such as capturing different patterns of click behavior, to produce a final decision on whether a click is genuine or fraudulent. Additionally, it integrates

explainable AI (XAI) techniques like SHAP values to provide transparency and highlight which features contribute most to each prediction.

7.1.5 EVALUATION MODULE:

To improve robustness and accuracy, predictions from multiple trained models are aggregated using ensemble techniques. This module combines the strengths of individual classifiers, such as capturing different patterns of click behavior, to produce a final decision on whether a click is genuine or fraudulent. Additionally, it integrates explainable AI (XAI) techniques like SHAP values to provide transparency and highlight which features contribute most to each prediction.

7.1.6 DEPLOYMENT MODULE:

This module handles the deployment of trained models and integrates them into a web-based dashboard for real-time predictions. Users can input click data and instantly receive predictions with explanations of feature contributions. The dashboard also features an interactive and visually engaging interface, with neon-glow cards and floating shapes, providing a consistent and futuristic experience for monitoring ad click behavior.

7.1.7 CONTINUOUS LEARNING MODULE:

Finally, the continuous learning module ensures that the system adapts to evolving click fraud patterns. New data is periodically collected, and models are retrained to maintain high accuracy and reduce vulnerability to emerging fraudulent tactics. This module is essential for keeping the system effective over time in dynamic digital advertising environments.

7.2 ALGORITHM TECHNIQUES USED:

The Ad Click Fraud Detection System employs **supervised machine learning techniques** to classify ad clicks as either genuine or fraudulent. The primary algorithms used in this project are **Random Forest** and **XGBoost**, both of which are well-suited for handling high-dimensional data and capturing complex patterns in user behavior.

1. Random Forest is an ensemble learning algorithm that constructs multiple decision trees during training and combines their outputs to produce a final prediction. This approach improves classification accuracy, reduces overfitting, and handles both numerical and categorical features effectively. Random Forest is particularly useful in fraud detection due to its robustness against noisy data and its ability to model non-linear relationships among features.

2. XGBoost (Extreme Gradient Boosting) is a powerful boosting-based algorithm that builds decision trees sequentially, where each new tree focuses on correcting the errors made by previous ones. It offers high predictive performance, efficient computation, and built-in regularization to prevent overfitting. XGBoost is highly effective in identifying subtle fraud patterns and managing class imbalance in large-scale datasets.

In addition to these algorithms, the project incorporates **feature engineering techniques** such as temporal feature extraction (hour, weekday, minute), behavioral analysis (click gaps and IP click counts), and data preprocessing methods including missing value imputation and feature scaling. Model evaluation techniques such as confusion matrices and ROC-AUC analysis are used to assess performance and select the best model for deployment.

CHAPTER 8

CONCLUSION AND FUTURE ENHANCEMENT

8.1 CONCLUSION:

The Ad Click Fraud Detection System developed in this project presents a reliable and intelligent solution to one of the major challenges faced by modern digital advertising platforms. By utilizing advanced feature engineering techniques and supervised machine learning algorithms such as Random Forest and XGBoost, the system effectively learns complex click behavior patterns and accurately distinguishes between genuine user interactions and fraudulent clicks generated by bots or malicious sources.

The system implementation combines data preprocessing, temporal and IP-based feature extraction, model training, evaluation, and deployment within a unified and well-structured framework. The integration of visualization techniques, including confusion matrices and ROC curves, supports transparent performance analysis and helps validate model effectiveness. Experimental results demonstrate strong predictive performance across multiple evaluation metrics such as accuracy, precision, recall, and AUC, indicating the robustness of the proposed approach.

In conclusion, the Ad Click Fraud Detection System offers a scalable, adaptable, and practical solution for real-world deployment in online advertising environments. Its modular architecture enables seamless integration with existing ad management and analytics platforms, while its machine learning-driven design allows continuous improvement against evolving fraud strategies. The project establishes a strong foundation for future enhancements, including real-time detection and advanced analytics, contributing significantly to improved trust, efficiency, and cost optimization in digital advertising ecosystems.

8.2 FUTURE ENHANCEMENT:

The Ad Click Fraud Detection System can be further enhanced in several ways to improve accuracy, scalability, and adaptability to evolving fraud patterns. One important future enhancement is the integration of real-time streaming data processing using technologies such as Apache Kafka or Spark Streaming. This would allow the system to detect fraudulent clicks instantly as they occur, rather than relying only on batch processing. Another potential improvement is the use of advanced deep learning models and hybrid approaches, such as combining machine learning with graph-based techniques to identify coordinated bot networks and click farms.

Incorporating behavioral analytics, including mouse movement patterns and session-level interaction features, can further strengthen fraud detection by capturing subtle human versus bot behavior. Additionally, applying explainable AI techniques such as SHAP or LIME can improve transparency and help advertisers understand why certain clicks are classified as fraudulent.

Future enhancements may also include deploying the system on cloud platforms to support large-scale ad networks and improve scalability. Developing a more interactive dashboard with real-time alerts, detailed visual analytics, and automated reporting would enhance usability for advertisers and fraud analysts. Continuous model re-training and adaptive learning mechanisms can be introduced to ensure the system remains effective against new and evolving fraud strategies. Overall, these enhancements would make the system more robust, intelligent, and suitable for real-world commercial deployment.

APPENDIX - 1

SOURCE CODE

[1]

```
import os
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from datetime import datetime
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import (
```

```
    confusion_matrix, classification_report,
```

```
    roc_curve, auc
```

```
)
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.compose import ColumnTransformer
```

```
from sklearn.impute import SimpleImputer
```

```
import joblib
```

[2]

try:

```
from xgboost import XGBClassifier
```

```
xgb_available = True
```

except:

```
xgb_available = False
```

```
print("WARNING: XGBoost is not installed. To install: pip install xgboost")
```

[3]

```
df = pd.read_csv(r"C:\Users\harik\OneDrive\Music\project\DATASET\sam-  
pled_train_dataset.csv")
```

```
print(df.head())
```

[4]

```
# Load Data
```

```
# -----
```

```
df = pd.read_csv(r"C:\Users\harik\OneDrive\Music\project\DATASET\sam-  
pled_train_dataset.csv")
```

```
test_df = pd.read_csv(r"C:\Users\harik\OneDrive\Music\project\DATASET\sam-  
pled_test_dataset.csv")
```

```
print("Train:", df.shape)
```

```
print("Test:", test_df.shape)
```

[5]

```
df.describe()
```

[6]

```
df.head()
```

[7]

```
df.isnull()
```

[8]

```
def feature_engineering(df):
```

```
    df = df.copy()
```

```
    df["click_time"] = pd.to_datetime(df["click_time"])
```

```
    df["hour"] = df["click_time"].dt.hour
```

```
    df["day"] = df["click_time"].dt.day
```

```
    df["weekday"] = df["click_time"].dt.weekday
```

```
    df["minute"] = df["click_time"].dt.minute
```

```
    df["epoch"] = df["click_time"].astype('int64') // 1e9
```

```
    df = df.sort_values(["ip", "click_time"])
```

```
    df["prev_epoch"] = df.groupby("ip")["epoch"].shift(1)
```

```
    df["click_gap"] = df["epoch"] - df["prev_epoch"]
```

```
    df["click_gap"] = df["click_gap"].fillna(0)
```

```
    df["ip_click_count"] = df["ip"].map(df["ip"].value_counts().to_dict())
```

```
df["is_night"] = ((df["hour"] >= 0) & (df["hour"] <= 6)).astype(int)
```

```
return df
```

```
df = feature_engineering(df)
```

```
test_df = feature_engineering(test_df)
```

```
[9]
```

```
plt.figure(figsize=(8,4))
```

```
sns.histplot(df["hour"], bins=24, kde=True)
```

```
plt.title("Hour-wise Click Distribution")
```

```
plt.xlabel("Hour of Day")
```

```
plt.ylabel("Click Count")
```

```
plt.show()
```

```
[10]
```

```
plt.figure(figsize=(8,4))
```

```
sns.histplot(df["ip_click_count"], bins=50, log_scale=True)
```

```
plt.title("IP Click Count Distribution")
```

```
plt.xlabel("Number of Clicks per IP")
```

```
plt.ylabel("Frequency")
```

```
plt.show()
```

[11]

```
plt.figure(figsize=(8,4))  
sns.histplot(df["click_gap"], bins=50, log_scale=True)  
plt.title("Click Gap Distribution")  
plt.xlabel("Time Gap Between Clicks (seconds)")  
plt.ylabel("Frequency")  
plt.show()
```

[12]

```
# Select Features  
  
# -----  
features = [  
    "ip", "app", "device", "os", "channel",  
    "hour", "weekday", "minute",  
    "click_gap", "ip_click_count", "is_night"  
]
```

[13]

```
# -----  
  
# Train-Test Split  
  
# -----  
X = df[features]  
y = df["is_attributed"].astype(int)  
  
X_train, X_valid, y_train, y_valid = train_test_split(
```

```

X, y, test_size=0.3, stratify=y, random_state=42
)

[14]
# Preprocessing
# -----

num_features = ["hour", "weekday", "minute", "click_gap", "ip_click_count", "is_night"]
cat_features = ["ip", "app", "device", "os", "channel"]

numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, num_features),
    ("cat", categorical_transformer, cat_features)
])

```

[15]

```
# -----
```

```
# Model: RandomForest
```

```
# -----
```

```
rf_pipeline = Pipeline([
    ("pre", preprocessor),
    ("clf", RandomForestClassifier(
        n_estimators=250,
        max_depth=15,
        n_jobs=-1,
        random_state=42
    ))
])
```

```
rf_pipeline.fit(X_train, y_train)
```

```
print("RandomForest trained!")
```

[16]

```
# =====
```

```
# 7. XGBOOST
```

```
# =====
```

```
if xgb_available:
```

```
    xgb_pipeline = Pipeline([
        ("pre", preprocessor),
        ("clf", XGBClassifier(
```



```

        n_estimators=300,
        max_depth=8,
        learning_rate=0.1,
        subsample=0.8,
        colsample_bytree=0.8,
        eval_metric="logloss"
    ))
])
xgb_pipeline.fit(X_train, y_train)
print("XGBoost trained!")

```

[17]

8. EVALUATION FUNCTION

=====

```
def evaluate(model, name):
```

```
    print(f"\n=== {name} Report ===")
```

```
    y_pred = model.predict(X_valid)
```

```
    y_prob = model.predict_proba(X_valid)[:,-1]
```

```
    print(classification_report(y_valid, y_pred))
```

```
    cm = confusion_matrix(y_valid, y_pred)
```

```
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
```

```
    plt.title(f'{name} Confusion Matrix')
```

```
    plt.show()
```

```

fpr, tpr, _ = roc_curve(y_valid, y_prob)

auc_score = auc(fpr, tpr)

plt.plot(fpr, tpr, label=f"AUC={auc_score:.4f}")
plt.plot([0,1],[0,1],"k--")
plt.title(f'{name} ROC Curve')
plt.legend()
plt.show()

return auc_score

```

[18]

```

import sklearn

!pip install xgboost

import xgboost

```

```

print("sklearn:", sklearn.__version__)
print("xgboost:", xgboost.__version__)

```

[19]

```

# 9. MODEL EVALUATION

# =====

rf_auc = evaluate(rf_pipeline, "RandomForest")

```

```
if xgb_available:
    xgb_auc = evaluate(xgb_pipeline, "XGBoost")
else:
    xgb_auc = -1
```

[20]

```
!pip install xgboost
```

[21]

```
best_model = rf_pipeline if rf_auc >= xgb_auc else xgb_pipeline
best_name = "RandomForest" if best_model is rf_pipeline else "XGBoost"

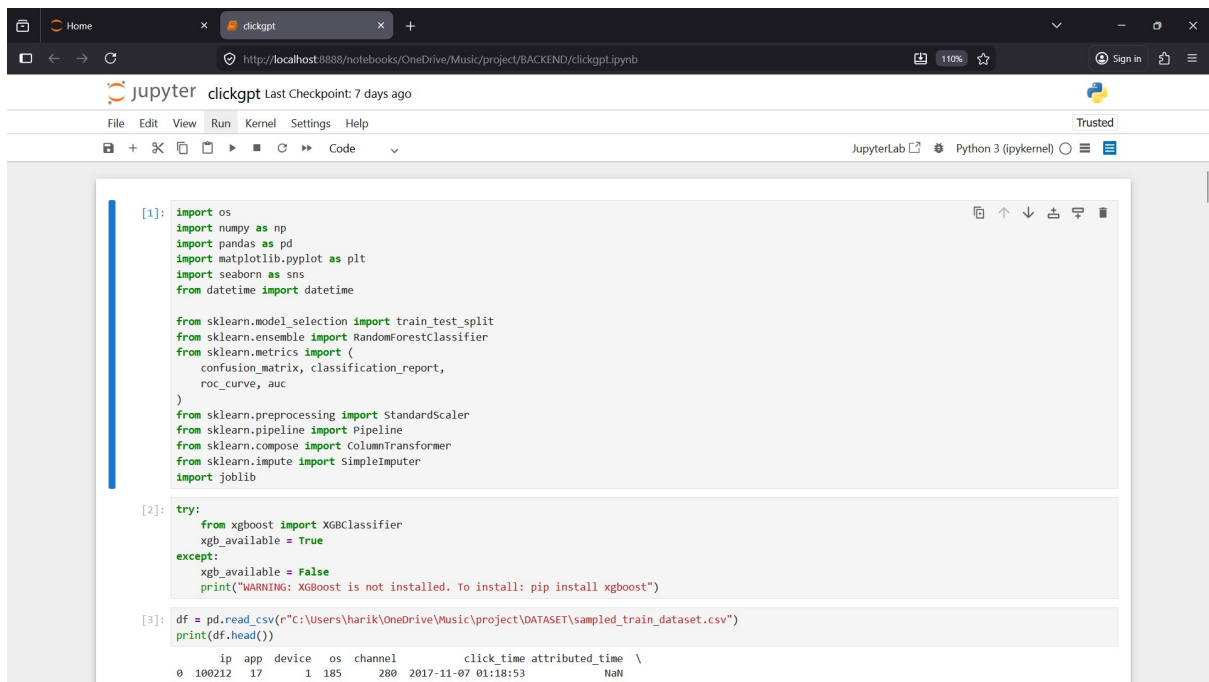
print("Best model =", best_name)
```

[22]

```
os.makedirs("./model", exist_ok=True)
joblib.dump(best_model, "./model/model.pkl")
joblib.dump(features, "./model/features.pkl")

print("Model saved successfully!")
```

JUPYTER NOTEBOOK OUTPUT:



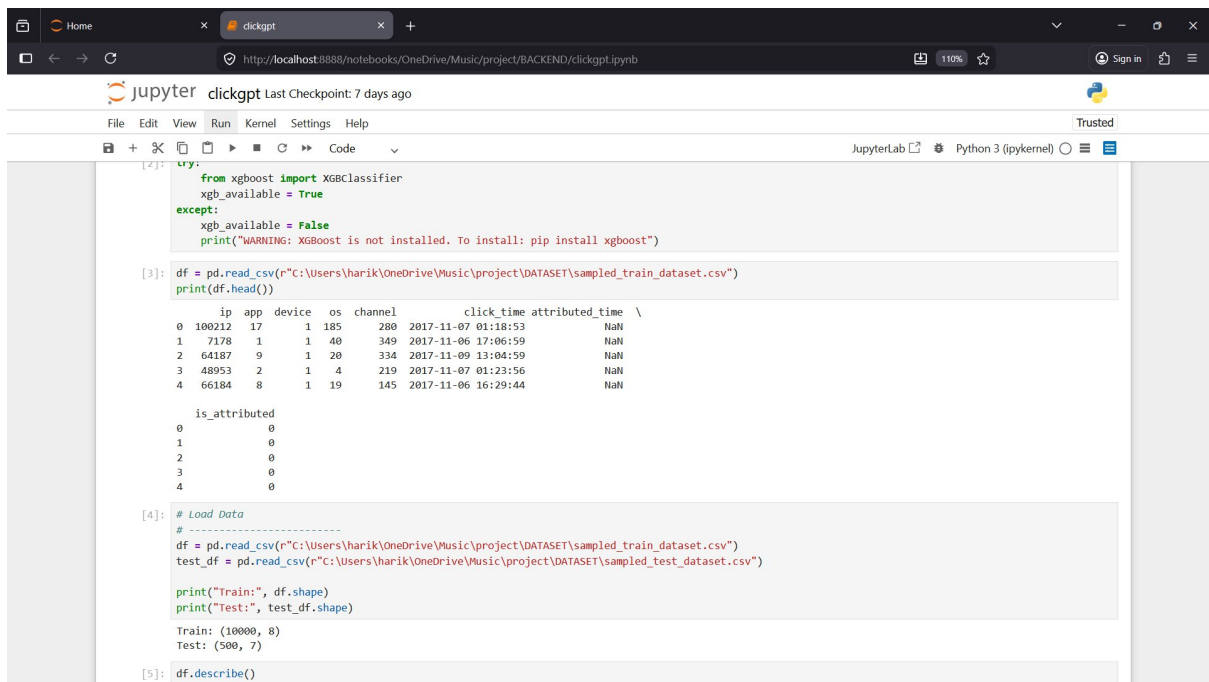
```
[1]: import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    confusion_matrix, classification_report,
    roc_curve, auc
)
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
import joblib

[2]: try:
    from xgboost import XGBClassifier
    xgb_available = True
except:
    xgb_available = False
    print("WARNING: XGBoost is not installed. To install: pip install xgboost")

[3]: df = pd.read_csv(r"C:\Users\harik\OneDrive\Music\project\DATASET\sampld_train_dataset.csv")
print(df.head())
```

	ip	app	device	os	channel	click_time	attributed_time	\
0	100212	17	1	185	280	2017-11-07 01:18:53	NaN	



```
[2]: try:
    from xgboost import XGBClassifier
    xgb_available = True
except:
    xgb_available = False
    print("WARNING: XGBoost is not installed. To install: pip install xgboost")

[3]: df = pd.read_csv(r"C:\Users\harik\OneDrive\Music\project\DATASET\sampld_train_dataset.csv")
print(df.head())
```

	ip	app	device	os	channel	click_time	attributed_time	\
0	100212	17	1	185	280	2017-11-07 01:18:53	NaN	
1	7178	1	1	40	349	2017-11-06 17:06:59	NaN	
2	64187	9	1	20	334	2017-11-09 13:04:59	NaN	
3	48953	2	1	4	219	2017-11-07 01:23:56	NaN	
4	66184	8	1	19	145	2017-11-06 16:29:44	NaN	

```
is_attributed
0      0
1      0
2      0
3      0
4      0

[4]: # Load Data
# -----
df = pd.read_csv(r"C:\Users\harik\OneDrive\Music\project\DATASET\sampld_train_dataset.csv")
test_df = pd.read_csv(r"C:\Users\harik\OneDrive\Music\project\DATASET\sampld_test_dataset.csv")

print("Train:", df.shape)
print("Test:", test_df.shape)

Train: (10000, 8)
Test: (500, 7)

[5]: df.describe()
```

clickgpt Last Checkpoint: 7 days ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```

Train: (10000, 8)
Test: (500, 7)

[5]: df.describe()

```

	ip	app	device	os	channel	is_attributed
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	89111.502600	12.513600	31.315500	24.592900	257.55810	0.002600
std	63260.753753	15.568245	304.763638	62.219433	130.22617	0.050926
min	45.000000	0.000000	0.000000	0.000000	3.000000	0.000000
25%	42200.000000	3.000000	1.000000	13.000000	135.00000	0.000000
50%	80779.500000	12.000000	1.000000	18.000000	245.00000	0.000000
75%	119320.000000	15.000000	1.000000	19.000000	377.00000	0.000000
max	364758.000000	525.000000	3866.000000	866.000000	497.00000	1.000000

```

[6]: df.head()

```

	ip	app	device	os	channel	click_time	attributed_time	is_attributed
0	100212	17	1	185	280	2017-11-07 01:18:53	NaN	0
1	7178	1	1	40	349	2017-11-06 17:06:59	NaN	0
2	64187	9	1	20	334	2017-11-09 13:04:59	NaN	0
3	48953	2	1	4	219	2017-11-07 01:23:56	NaN	0
4	66184	8	1	19	145	2017-11-06 16:29:44	NaN	0

clickgpt Last Checkpoint: 7 days ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```

[7]: df.isnull()

```

	ip	app	device	os	channel	click_time	attributed_time	is_attributed
0	False	False	False	False	False	False	True	False
1	False	False	False	False	False	False	True	False
2	False	False	False	False	False	False	True	False
3	False	False	False	False	False	False	True	False
4	False	False	False	False	False	False	True	False
...	...	...	...	...	...	...	...	...
9995	False	False	False	False	False	False	True	False
9996	False	False	False	False	False	False	True	False
9997	False	False	False	False	False	False	True	False
9998	False	False	False	False	False	False	True	False
9999	False	False	False	False	False	False	True	False

10000 rows x 8 columns

```

[8]: def feature_engineering(df):
    df = df.copy()
    df["click_time"] = pd.to_datetime(df["click_time"])

    df["hour"] = df["click_time"].dt.hour
    df["day"] = df["click_time"].dt.day
    df["weekday"] = df["click_time"].dt.weekday
    df["minute"] = df["click_time"].dt.minute

```

```
def feature_engineering(df):
    df = df.copy()
    df["click_time"] = pd.to_datetime(df["click_time"])

    df["hour"] = df["click_time"].dt.hour
    df["day"] = df["click_time"].dt.day
    df["weekday"] = df["click_time"].dt.weekday
    df["minute"] = df["click_time"].dt.minute

    df["epoch"] = df["click_time"].astype('int64') // 1e9

    df = df.sort_values(["ip", "click_time"])
    df["prev_epoch"] = df.groupby("ip")["epoch"].shift(1)
    df["click_gap"] = df["epoch"] - df["prev_epoch"]
    df["click_gap"] = df["click_gap"].fillna(0)

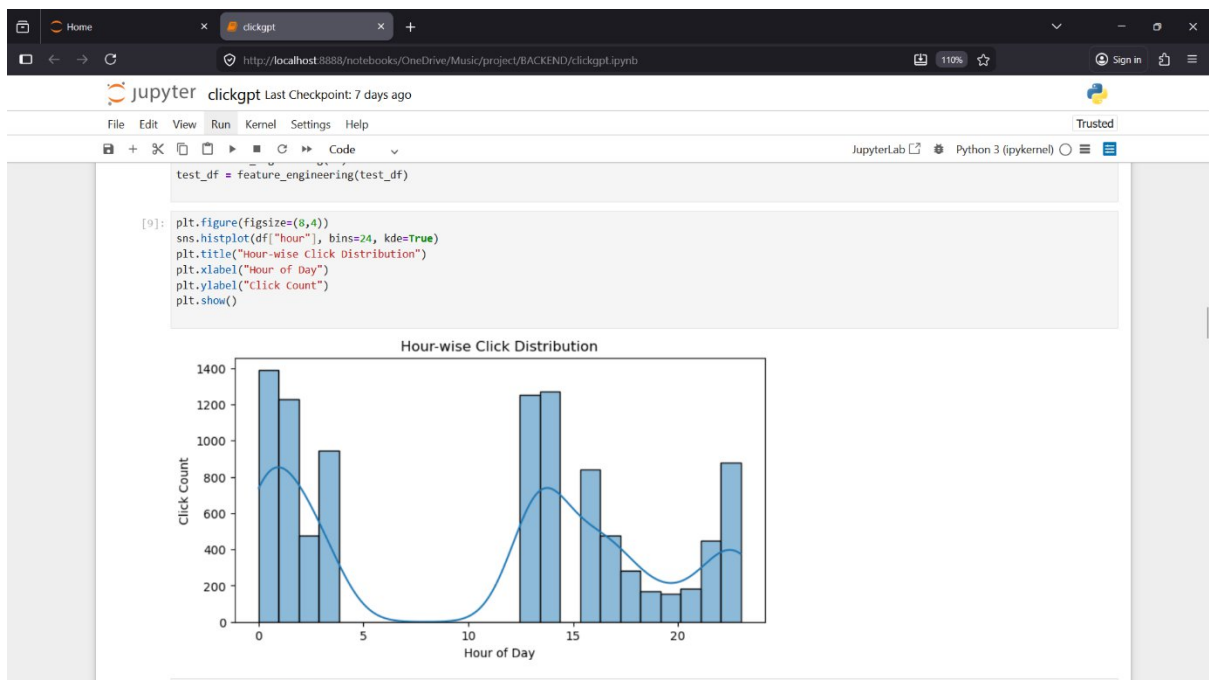
    df["ip_click_count"] = df["ip"].map(df["ip"].value_counts().to_dict())

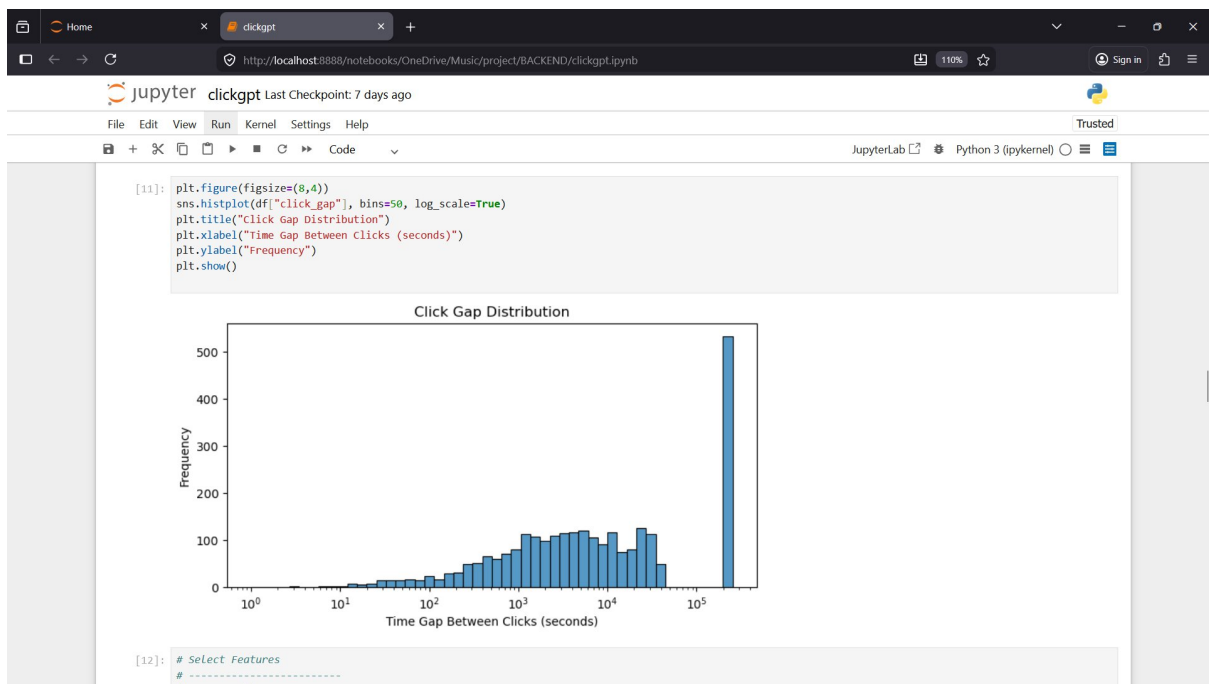
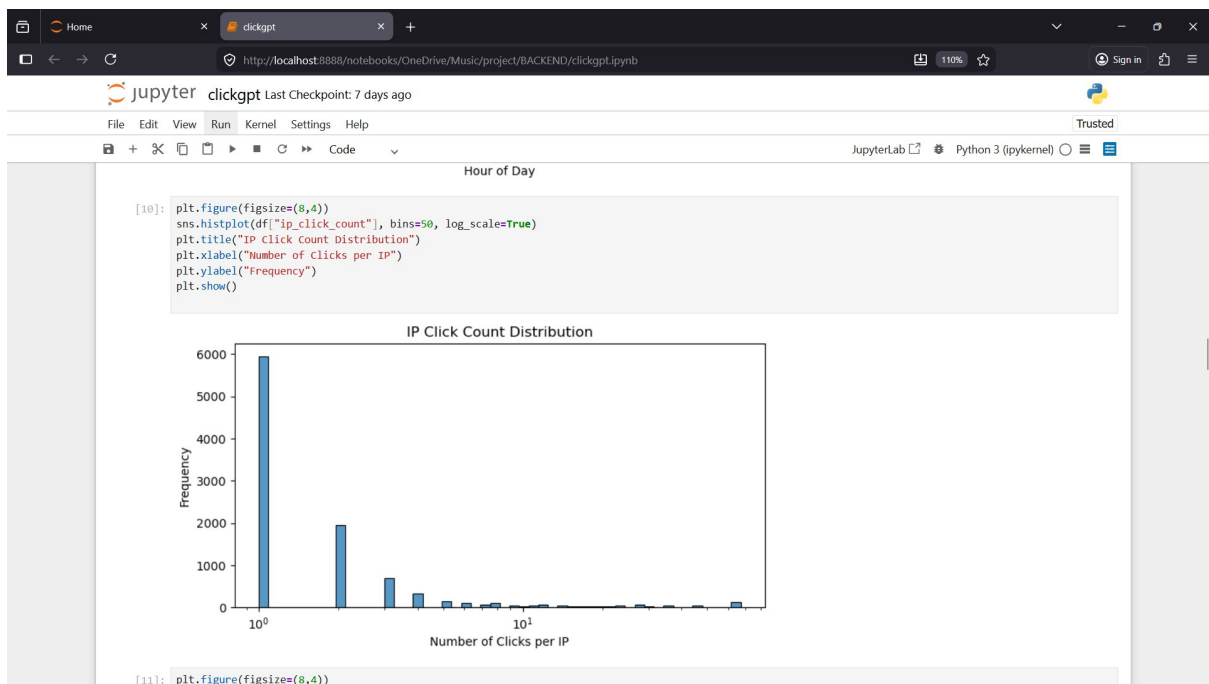
    df["is_night"] = ((df["hour"] >= 0) & (df["hour"] <= 6)).astype(int)

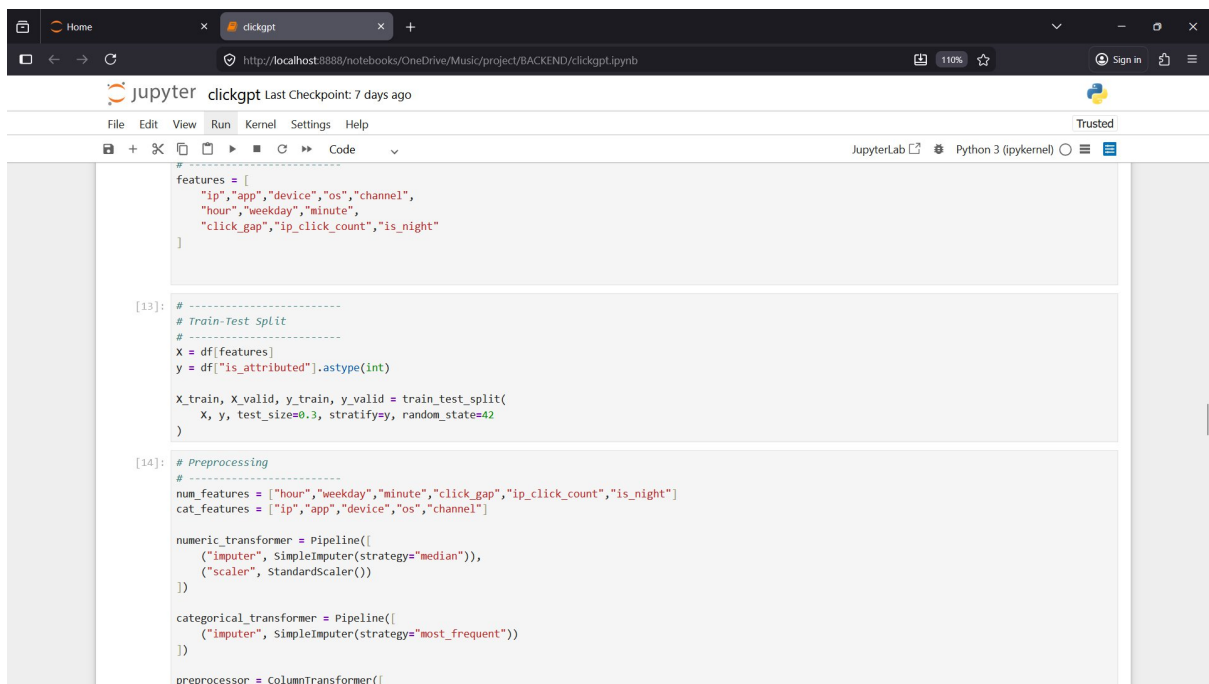
    return df

df = feature_engineering(df)
test_df = feature_engineering(test_df)

plt.figure(figsize=(8,4))
sns.histplot(df["hour"], bins=24, kde=True)
plt.title("Hour-wise Click Distribution")
plt.xlabel("Hour of Day")
plt.ylabel("Click Count")
plt.show()
```

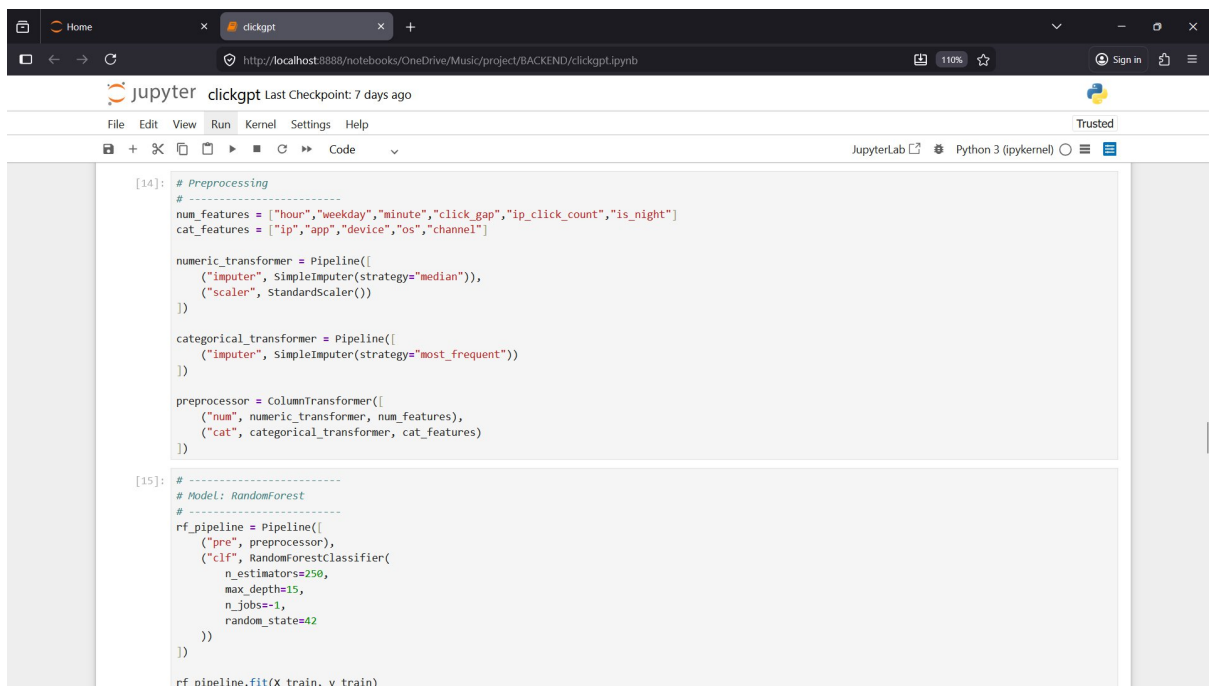






The screenshot shows a JupyterLab window with a browser address bar at `http://localhost:8888/notebooks/OneDrive/Music/project/BACKEND/clickgpt.ipynb`. The interface includes a top menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and execution. The code editor displays the following Python code:

```
# -----  
features = [  
    "ip", "app", "device", "os", "channel",  
    "hour", "weekday", "minute",  
    "click_gap", "ip_click_count", "is_night"  
]  
  
[13]: # -----  
# Train-Test Split  
# -----  
X = df[features]  
y = df["is_attributed"].astype(int)  
  
X_train, X_valid, y_train, y_valid = train_test_split(  
    X, y, test_size=0.3, stratify=y, random_state=42  
)  
  
[14]: # Preprocessing  
# -----  
num_features = ["hour", "weekday", "minute", "click_gap", "ip_click_count", "is_night"]  
cat_features = ["ip", "app", "device", "os", "channel"]  
  
numeric_transformer = Pipeline([  
    ("imputer", SimpleImputer(strategy="median")),  
    ("scaler", StandardScaler())  
)  
  
categorical_transformer = Pipeline([  
    ("imputer", SimpleImputer(strategy="most_frequent"))  
)  
  
preprocessor = ColumnTransformer([
```



The screenshot shows the continuation of the JupyterLab window. The code editor displays the following Python code:

```
preprocessor = ColumnTransformer([  
    ("num", numeric_transformer, num_features),  
    ("cat", categorical_transformer, cat_features)  
)  
  
[14]: # Preprocessing  
# -----  
num_features = ["hour", "weekday", "minute", "click_gap", "ip_click_count", "is_night"]  
cat_features = ["ip", "app", "device", "os", "channel"]  
  
numeric_transformer = Pipeline([  
    ("imputer", SimpleImputer(strategy="median")),  
    ("scaler", StandardScaler())  
)  
  
categorical_transformer = Pipeline([  
    ("imputer", SimpleImputer(strategy="most_frequent"))  
)  
  
preprocessor = ColumnTransformer([  
    ("num", numeric_transformer, num_features),  
    ("cat", categorical_transformer, cat_features)  
)  
  
[15]: # -----  
# Model: RandomForest  
# -----  
rf_pipeline = Pipeline([  
    ("pre", preprocessor),  
    ("clf", RandomForestClassifier(  
        n_estimators=250,  
        max_depth=15,  
        n_jobs=-1,  
        random_state=42  
    ))  
)  
  
rf_pipeline.fit(X_train, y_train)
```


The screenshot shows a JupyterLab window with a single code cell. The code defines a Random Forest pipeline and trains it. The output shows the model is trained successfully.

```
[15]: # -----  
# Model: RandomForest  
# -----  
rf_pipeline = Pipeline(  
    ("pre", preprocessor),  
    ("clf", RandomForestClassifier(  
        n_estimators=250,  
        max_depth=15,  
        n_jobs=-1,  
        random_state=42  
    ))  
)  
  
rf_pipeline.fit(X_train, y_train)  
print("RandomForest trained!")  
  
RandomForest trained!
```

```
[16]: # =====  
# 7. XGBOOST  
# =====  
if xgb_available:  
    xgb_pipeline = Pipeline(  
        ("pre", preprocessor),  
        ("clf", XGBClassifier(  
            n_estimators=300,  
            max_depth=8,  
            learning_rate=0.1,  
            subsample=0.8,  
            colsample_bytree=0.8,  
            eval_metric="logloss"  
        ))  
    )  
    xgb_pipeline.fit(X_train, y_train)  
    print("XGBoost trained!")
```

The screenshot shows a JupyterLab window with two code cells. The first cell trains an XGBoost model. The second cell defines an evaluation function that prints the classification report and shows a confusion matrix heatmap.

```
[16]: # =====  
# 7. XGBOOST  
# =====  
if xgb_available:  
    xgb_pipeline = Pipeline(  
        ("pre", preprocessor),  
        ("clf", XGBClassifier(  
            n_estimators=300,  
            max_depth=8,  
            learning_rate=0.1,  
            subsample=0.8,  
            colsample_bytree=0.8,  
            eval_metric="logloss"  
        ))  
    )  
    xgb_pipeline.fit(X_train, y_train)  
    print("XGBoost trained!")  
  
XGBoost trained!
```

```
[17]: # 8. EVALUATION FUNCTION  
# =====  
def evaluate(model, name):  
    print(f"\n--- (name) Report ---")  
    y_pred = model.predict(X_valid)  
    y_prob = model.predict_proba(X_valid)[:,1]  
  
    print(classification_report(y_valid, y_pred))  
  
    cm = confusion_matrix(y_valid, y_pred)  
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")  
    plt.title(f"(name) Confusion Matrix")  
    plt.show()
```

```

[17]: # 8. EVALUATION FUNCTION
# =====
def evaluate(model, name):
    print(f"\n=== (name) Report ===")
    y_pred = model.predict(X_valid)
    y_prob = model.predict_proba(X_valid)[:,1]

    print(classification_report(y_valid, y_pred))

    cm = confusion_matrix(y_valid, y_pred)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
    plt.title(f"(name) Confusion Matrix")
    plt.show()

    fpr, tpr, _ = roc_curve(y_valid, y_prob)
    auc_score = auc(fpr, tpr)

    plt.plot(fpr, tpr, label=f"AUC={auc_score:.4f}")
    plt.plot([0,1],[0,1], "k--")
    plt.title(f"(name) ROC Curve")
    plt.legend()
    plt.show()

    return auc_score

[18]: import sklearn
      !pip install xgboost
      import xgboost

      print("sklearn:", sklearn.__version__)
      print("xgboost:", xgboost.__version__)
  
```

```

[18]: import sklearn
      !pip install xgboost
      import xgboost

      print("sklearn:", sklearn.__version__)
      print("xgboost:", xgboost.__version__)

Requirement already satisfied: xgboost in d:\app\lib\site-packages (3.2.0)
Requirement already satisfied: numpy in d:\app\lib\site-packages (from xgboost) (2.3.5)
Requirement already satisfied: scipy in d:\app\lib\site-packages (from xgboost) (1.16.3)
sklearn: 1.7.2
xgboost: 3.2.0

[19]: # 9. MODEL EVALUATION
# =====
rf_auc = evaluate(rf_pipeline, "RandomForest")

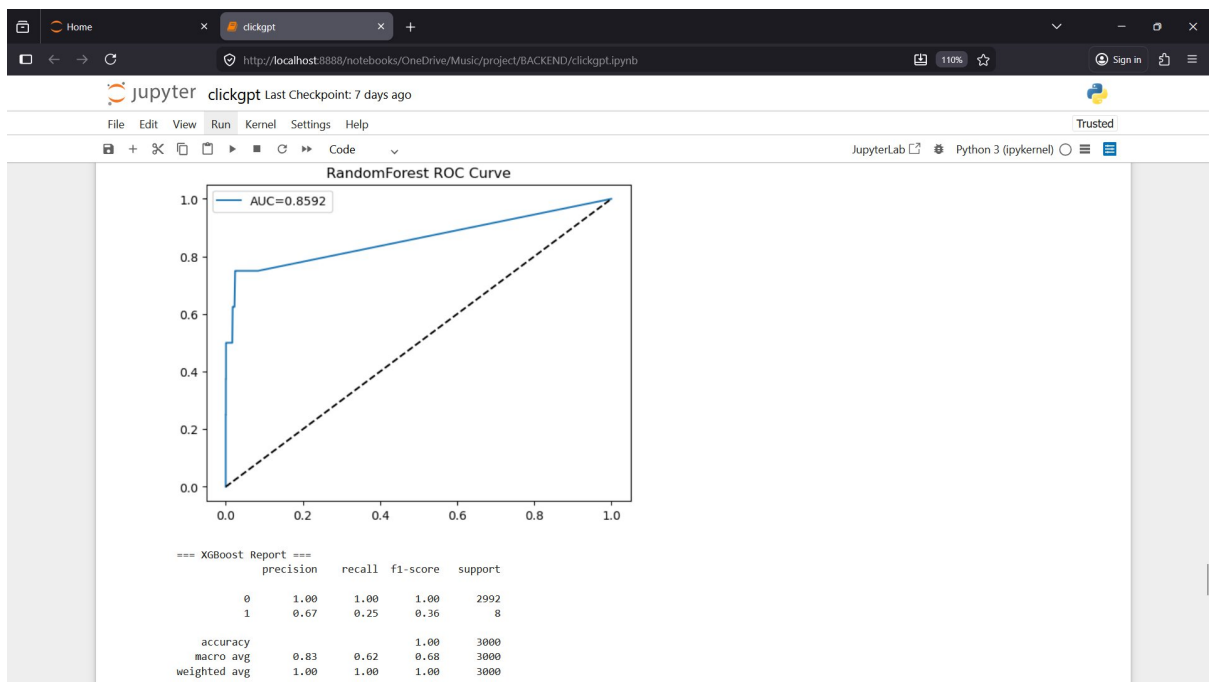
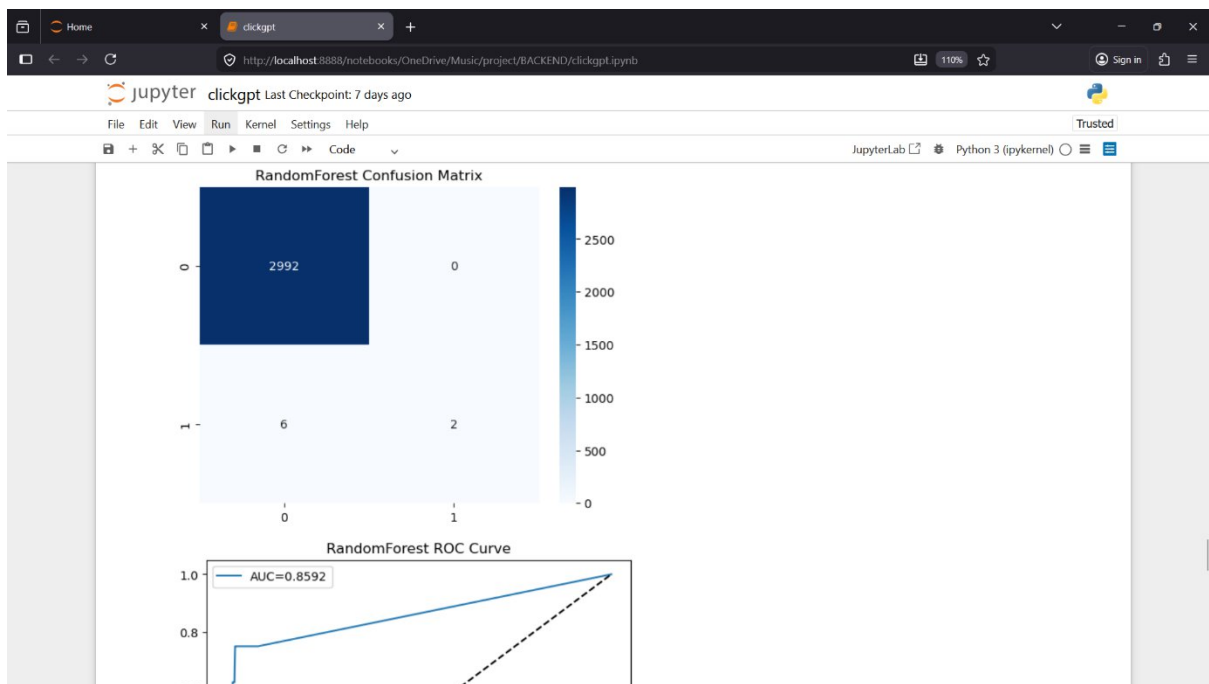
if xgb_available:
    xgb_auc = evaluate(xgb_pipeline, "XGBoost")
else:
    xgb_auc = -1

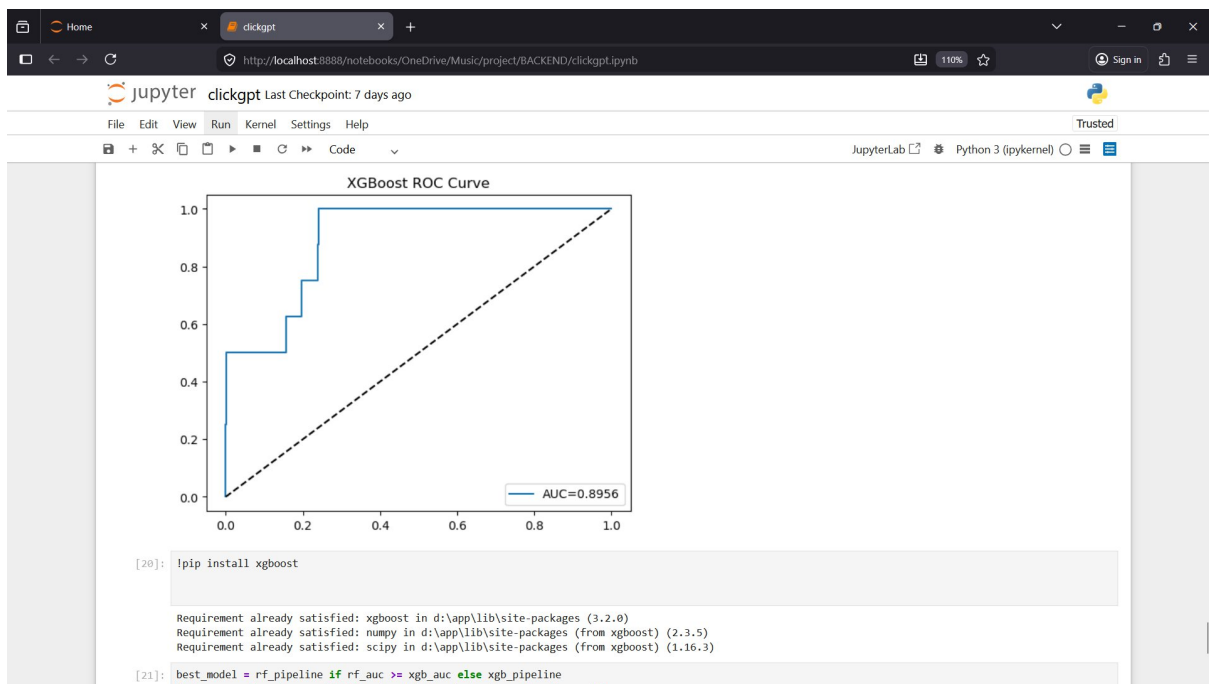
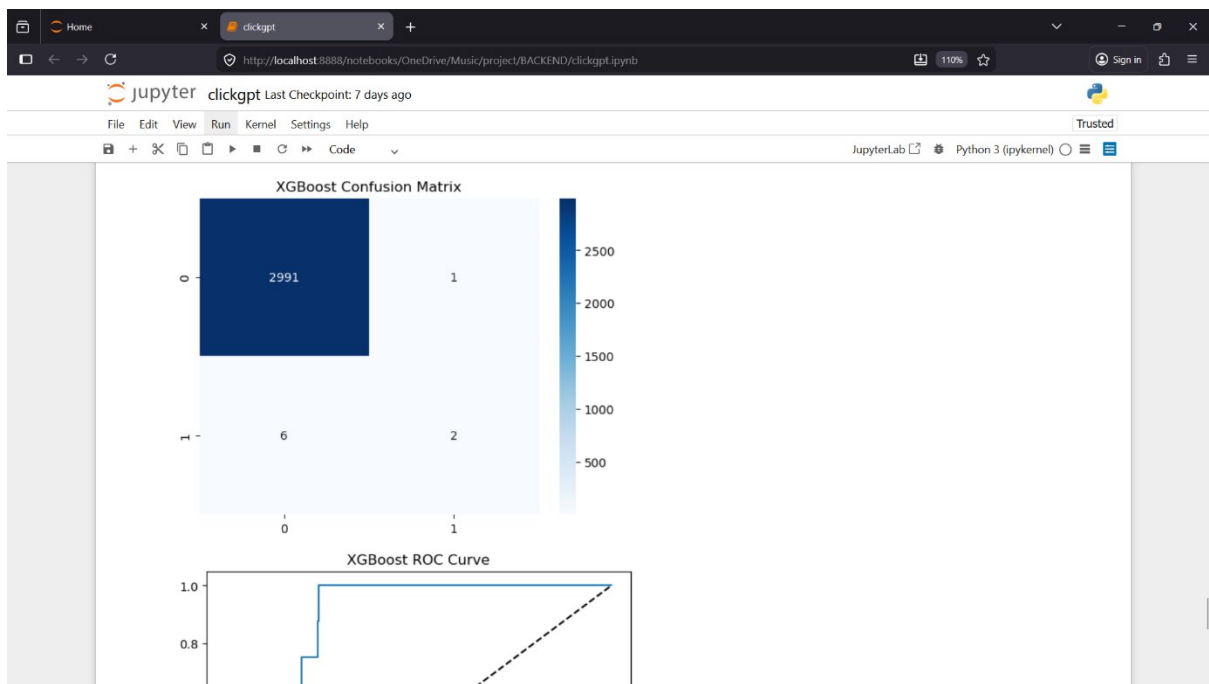
=== RandomForest Report ===
      precision    recall  f1-score   support

     0       1.00      1.00      1.00     2992
     1       1.00      0.25      0.40         8

 accuracy          1.00          0.62          0.70     3000
 macro avg          1.00          0.62          0.70     3000
 weighted avg          1.00          1.00          1.00     3000

RandomForest Confusion Matrix
  
```





```
[20]: !pip install xgboost

Requirement already satisfied: xgboost in d:\app\lib\site-packages (3.2.0)
Requirement already satisfied: numpy in d:\app\lib\site-packages (from xgboost) (2.3.5)
Requirement already satisfied: scipy in d:\app\lib\site-packages (from xgboost) (1.16.3)

[21]: best_model = rf_pipeline if rf_auc >= xgb_auc else xgb_pipeline
best_name = "RandomForest" if best_model is rf_pipeline else "XGBoost"

print("Best model =", best_name)

Best model = XGBoost

[22]: os.makedirs("./model", exist_ok=True)
joblib.dump(best_model, "./model/model.pkl")
joblib.dump(features, "./model/features.pkl")

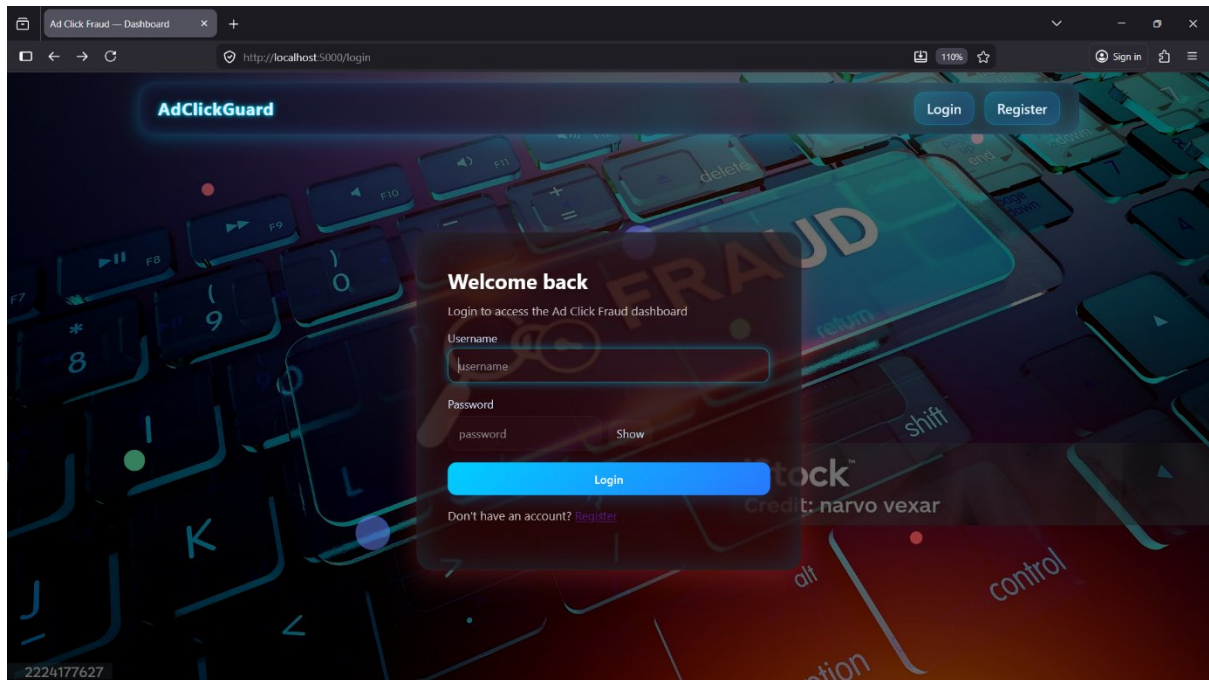
print("Model saved successfully!")
Model saved successfully!

[ ]:
```

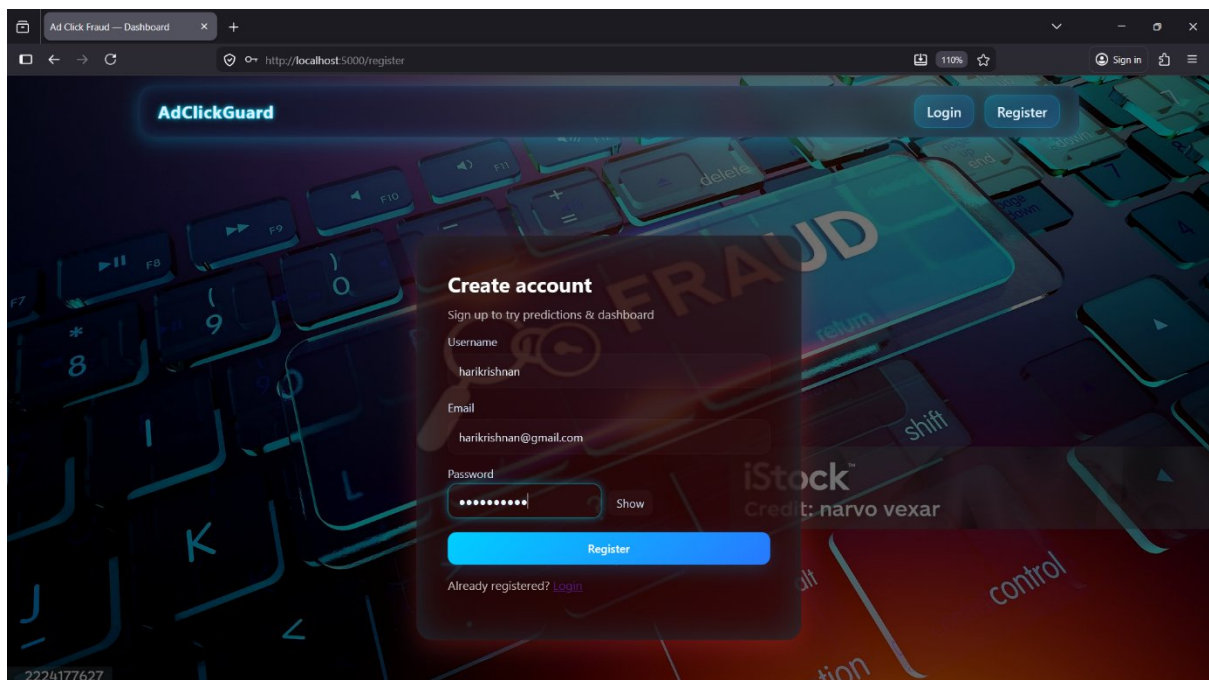
APPENDIX – 2

OUTPUT SCREENSHOTS:

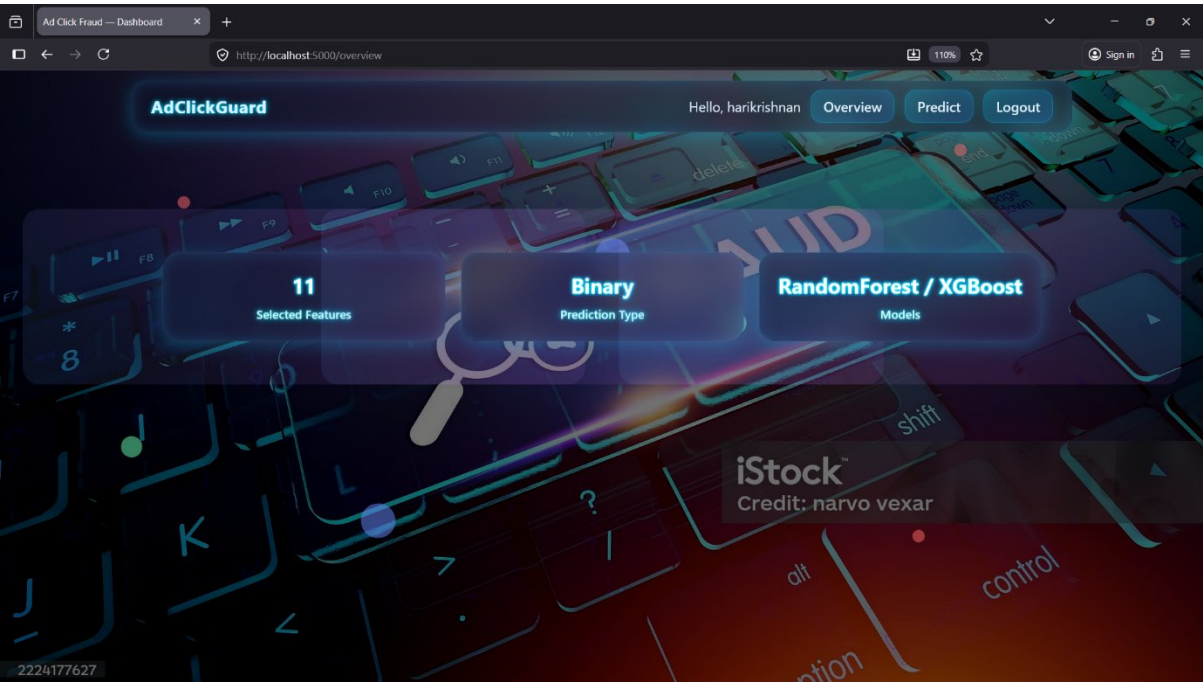
1. Login Page:



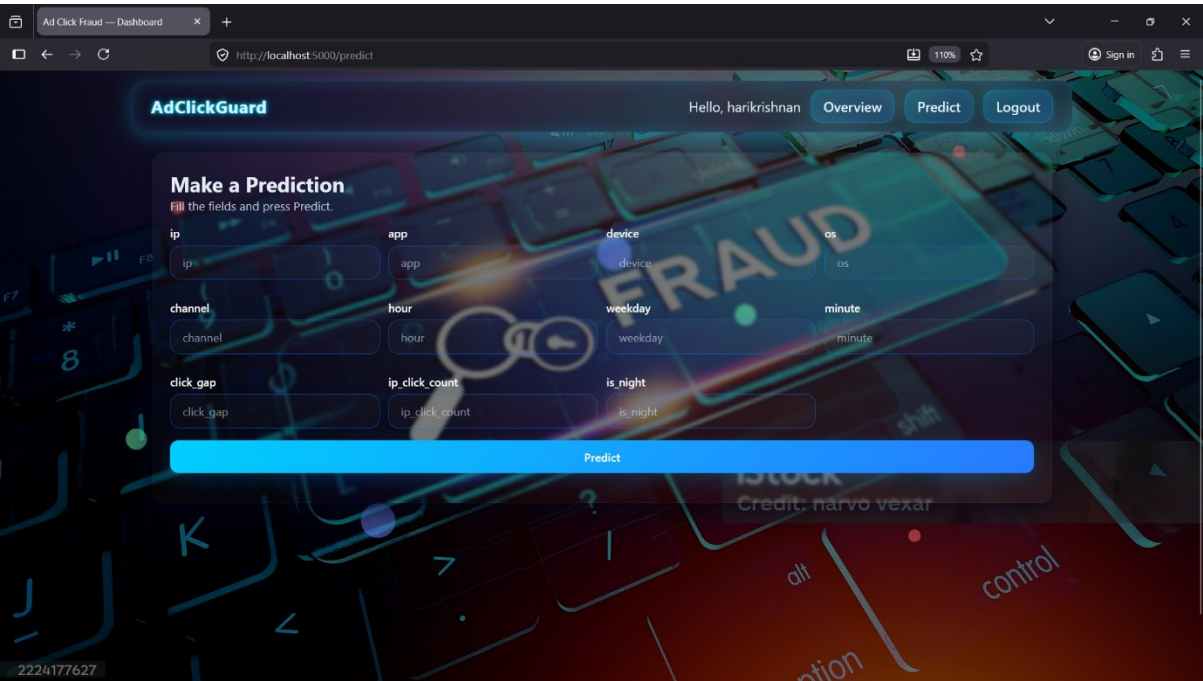
2. Registration Page:



3. Home Page:



4. Predictor Page:



5. Bot Click Sample:

AdClickGuard

Hello, hanikrishnan

Overview Predict Logout

Make a Prediction
Fill the fields and press Predict.

ip	app	device	os
9234	14	1	13
channel	hour	weekday	minute
480	hour	weekday	minute
click_gap	ip_click_count	is_night	
click_gap	ip_click_count	is_night	

Predict

6. Bot Click Prediction Message:

AdClickGuard

Hello, hanikrishnan

Overview Predict Logout

Make a Prediction
Fill the fields and press Predict.

Prediction: **Bot Click**

ip	app	device	os
ip	app	device	os
channel	hour	weekday	minute
channel	hour	weekday	minute
click_gap	ip_click_count	is_night	
click_gap	ip_click_count	is_night	

Predict

7. Human Click Sample:

AdClickGuard

Hello, hanikrishnan [Overview](#) [Predict](#) [Logout](#)

Make a Prediction

Fill the fields and press Predict.

ip	app	device	os
15696	35	1	19
channel	hour	weekday	minute
21	hour	weekday	minute
click_gap	ip_click_count	is_night	
click_gap	ip_click_count	is_night	

[Predict](#)

8. Human click Prediction Massage:

AdClickGuard

Hello, hanikrishnan [Overview](#) [Predict](#) [Logout](#)

Make a Prediction

Fill the fields and press Predict.

Prediction: Human Click

ip	app	device	os
ip	app	device	os
channel	hour	weekday	minute
channel	hour	weekday	minute
click_gap	ip_click_count	is_night	
click_gap	ip_click_count	is_night	

[Predict](#)

References:

- [1] A. Sharma, S. Singh, and V. K. Yadav, "Machine learning-based click fraud detection system for online advertising," *Indian Journal of Computer Science and Engineering*, vol. 10, no. 4, pp. 112–123, 2023.
- [2] R. Gupta and P. R. Patel, "Ensemble learning approach for ad click fraud detection in Indian digital markets," *International Journal of Intelligent Systems and Applications*, vol. 15, no. 6, pp. 55–65, 2024.
- [3] S. Kumar, A. Verma, and R. Singh, "Behavioral feature extraction for ad click fraud detection using supervised learning," *IEEE International Conference on Signal Processing, Computation, and Control*, pp. 211–218, 2024.
- [4] P. Mohanty and S. Rao, "Real-time click fraud detection using online learning algorithms," *Journal of Advanced Research in Dynamical & Control Systems*, vol. 14, no. 5, pp. 747–756, 2022.
- [5] A. Jain, K. D. Kumar, and M. Sharma, "Click fraud detection in digital advertising using feature non-linear patterns," *International Conference on Big Data Analytics and Intelligent Systems*, pp. 329–336, 2023.
- [6] M. Singh and R. Bhatia, "Lightweight feature engineering for click fraud detection on resource-constrained systems," *ACM India Joint International Conference on Data Science & Security*, pp. 99–108, 2024.
- [7] T. Das and A. K. Saha, "Hybrid anomaly and supervised model framework for detecting coordinated ad fraud," *Journal of Information Security and Applications*, vol. 65, pp. 1–15, 2024.
- [8] V. Sharma, A. Choudhary, and R. Agarwal, "Explainable machine learning for ad fraud detection using SHAP and ensemble models," *Indian Journal of Artificial Intelligence & Machine Learning*, vol. 2, no. 3, pp. 29–38, 2025.

- [9] R. Nair and S. Menon, “Deep learning based ad click fraud detection with temporal sequence modeling,” *International Conference on Artificial Intelligence & Data Science*, pp. 451–458, 2025.
- [10] K. Reddy and S. K. Mishra, “Adaptive thresholding and time-based features for real-time ad click fraud detection,” *IEEE Region 10 Symposium (TENSYP)*, pp. 101–108, 2023.